Option Pay

คู่มือ API v1

ตัวอย่างคำสั่ง curl และ Next.js แบบเต็มทุก endpoint · การเซ็นคำขอ · สิทธิ์และข้อผิดพลาด · การตรวจสลิป · Callback ที่ระบบส่งถึงร้าน

Merchant Portal https://optionpay.co/merchant/login
ฉบับออนไลน์ (HTML) Merchant Portal → คู่มือ → คู่มือ API v1
อัปเดต 18 ตุลาคม 2569

สารบัญอยู่หน้าถัดไป — กดหัวข้อเพื่อไปยังหน้านั้น · หรือใช้ Bookmarks ของโปรแกรมอ่าน PDF

สารบัญ

กดชื่อหัวข้อหรือ endpoint เพื่อไปยังหน้านั้น

1. เริ่มต้นใช้งาน 4
2. Secret key และ API key 6
3. การเซ็นคำขอ (X-Signature) 7
4. การตรวจสลิป 9
5. ส่งสลิป: timeout และการส่งซ้ำ 12
6. รายงานผลการตรวจสลิป 15
7. สิทธิ์และข้อผิดพลาด 24
8. Callback ที่ระบบส่งถึงร้าน (curl) 26
9. การยืนยันตัวตน (Authentication) 34
   - POST /auth/login 34
10. Payment 36
    - POST /payment/create 36
    - POST /payment/slip/create 41
    - POST /payment/slip/verify 43
    - POST /payment/:ref/slip 49
    - GET /payment/validation-report 51
    - GET /payment/order/:orderId 51
    - GET /payment/checkout/:transactionId 53
    - POST /payment/checkout/:transactionId/slip 54
    - GET /payment/qr/:token.png 56
    - POST /collect 56
11. Transactions (ประวัติ) 60
    - GET /transactions 60
    - GET /transactions/:id 62
    - PATCH /transactions/:id/cancel 64
12. Payout (โอนเงินออก) 67
    - POST /payout/create 67
    - GET /payout/:id 69
    - GET /payout/ref/:merchantRef 71
    - GET /payouts 72
    - POST /payout/bulk 74
13. Withdrawal Request (คำขอถอน) 78
    - POST /withdrawal/request/create 78
    - GET /withdrawal/requests 80
    - GET /withdrawal/request/:ref 82

14. Payment Accounts (บัญชีรับเงิน) 84
GET /payment/accounts 84
POST /payment/accounts 85
DELETE /payment/accounts/:id 87
15. Webhooks 90
GET /webhooks 90
POST /webhooks 91
16. Balance & Profile 95
GET /balance 95
GET /balance/line-summary 96
GET /profile 99
GET /dashboard 101
17. Sub-merchants (ร้านย่อย) 104
GET /sub-merchants 104
GET /sub-merchants/overview 110
GET /sub-merchants/:id 116
POST /sub-merchants 119
POST /sub-merchants/:id 122
POST /sub-merchants/:id/owner-password 123
GET /balance (X-Sub-Merchant) 125

เริ่มต้นใช้งาน

1. **รับ key** — หน้า ตั้งค่า → กด **สร้าง secret key** ได้ **API key** (`ak_live_…`) และ **secret key** (`sk_live_…`) ทันที
2. **เซ็นทุกคำขอ** — ส่ง `X-API-Key` , `X-Timestamp` , `X-Signature` (ดูหัวข้อ “การเซ็นคำขอ”)
3. **สร้างออเดอร์** — `POST /payment/create` แล้วพาลูกค้าไปที่ `checkoutUrl`
4. **รับผล** — ตั้ง webhook หรือเช็ค `GET /payment/order/:orderId`

ตัวอย่าง: สร้างออเดอร์ (CURL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792505"   # ปกติใช้ TS=$(date +%s)
BODY='{"amount":100,"orderId":"ORD-DOC-001","description":"สินค้า A","metadata":
{"senderAccountNumber":"1234567890","senderBankCode":"SCB","memberName":"สมชาย
ใจดี","customerId":"CUST-9"}}'
SIG=$(printf '%s' "$TS.POST./api/v1/payment/create.$BODY" | openssl dgst -
sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 75871079d00b21f2deb12dd833bd4551b5a41d85a9833e942c117f4751deb9cc

curl -X POST "https://optionpay.co/api/v1/payment/create" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "orderIdGenerated": false,
  "transaction": {
    "id": "d3m00000000000000000000000000001",
    "ref": "TXN100011",
    "merchantId": "6",
    "orderId": "ORD-DOC-001",
    "amount": 100,
    "fee": 1.5,
    "netAmount": 98.5,
    "status": "PENDING",
    "source": "API",
    "paymentMethod": "BANK_TRANSFER",
    "description": "สินค้า A",
    "qrCode": null,
    "qrCodeUrl": null,
    "qrExpiry": "2026-10-12T08:23:26.830Z",
    "checkoutUrl": "https://optionpay.co/c?t=d3m00000000000000000000000000001",
    "createdAt": "2026-10-12T08:08:26.830Z",
    "paidAt": null,
    "copy": {
```

```
      "amount": "100.00"
    }
  },
  "selectedAccount": {
    "id": "8",
    "type": "BANK",
    "accountNumber": "1234567890",
    "accountName": "บริษัท ตัวอย่าง จำกัด",
    "bankCode": "SCB",
    "bankName": "ธนาคารไทยพาณิชย์"
  }
}
```

Secret key และ API key

	รูปแบบ	ส่งไปกับคำขอ?	ใช้ทำอะไร
API key	`ak_live_…`	ส่ง — header X-API-Key	บอกว่าใครเรียก (ร้าน + ผู้ใช้)
Secret key	`sk_live_…`	**ห้ามส่ง**	เซ็นทุกคำขอ (X-Signature) ของ API key ผู้ใช้
Signing secret	`ss_live_…`	**ห้ามส่ง**	เซ็นคำขอของ API key ระดับร้าน (ที่ออกในการ์ด API Keys หรือโดยผู้ดูแล)

1. Merchant Portal → ตั้งค่า → **Secret key และ API key** → กด **สร้าง secret key** — ได้ทั้งคู่ทันที แสดงแบบเต็มครั้งแรก
2. ภายหลังแสดงแบบปิดบางส่วน — กด **ดู** + ยืนยันชื่อผู้ใช้/รหัสผ่าน เพื่อแสดงแบบเต็ม 1 นาที
3. กด **สร้างใหม่** เพื่อเปลี่ยนทั้งคู่ (อันเดิมใช้ไม่ได้ทันที) · API key มีสิทธิ์ตามระดับผู้ใช้
4. API key ระดับร้าน (การ์ด **API Keys**) ออกพร้อม **signing secret** แสดงครั้งเดียว — ใช้ทั้งคู่แทน API key + secret key ของผู้ใช้

> ขอ key ได้ใน Merchant Portal เท่านั้น (`/auth/login` เรียกจากภายนอกได้ `403 LOGIN_PORTAL_ONLY`) · key รูปแบบเดิม (`sk_live_` เป็น API key, `ms_live_`) ถูกยกเลิกทั้งหมดแล้ว · เก็บ secret ไว้ในเซิร์ฟเวอร์ของร้านเท่านั้น

การเซ็นคำขอ (X-Timestamp + X-Signature)

บังคับทุก endpoint ที่ใช้ `X-API-Key` — ไม่มีเปิด/ปิด

1. `X-Timestamp` = เวลาปัจจุบันแบบ unix วินาที (คลาดได้ไม่เกิน 5 นาที)
2. `X-Signature` = **HMAC-SHA256**(secret, `<timestamp>.<METHOD>.<path>.<body>`) แบบ hex ตัวเล็ก — secret = secret key ของผู้ใช้ (`sk_live_`) หรือ signing secret ของ key ระดับร้าน (`ss_live_`)
3. ส่ง `X-API-Key` + `X-Timestamp` + `X-Signature` และ body เดียวกับที่เซ็น — **ห้ามส่ง secret**

สตริงที่เซ็น

```
<timestamp>.<METHOD>.<path รวม query>.<body>

ตัวอย่าง:
1791792505.POST./api/v1/payment/create.{"amount":100,"orderId":"ORD-DOC-
001","description":"สินค้า A","metadata":
{"senderAccountNumber":"1234567890","senderBankCode":"SCB","memberName":"สมชาย
ใจดี","customerId":"CUST-9"}}
1791792520.GET./api/v1/transactions?limit=2&offset=0&status=PENDING.        ←
GET ไม่มี body ลงท้ายด้วย "."
```

NEXT.JS — HELPER ใช้ซ้ำได้ทุก ENDPOINT

```
// lib/optionpay.ts — ใช้ซ้ำได้ทุก endpoint (ฝั่งเซิร์ฟเวอร์เท่านั้น)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ak_live_…
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // sk_live_… (key ระดับร้านใช้
ss_live_…) — ห้ามส่ง

// เซ็นใหม่ทุกครั้งที่เรียก (timestamp ใหม่) — ส่งซ้ำได้โดยไม่ติด SIGNATURE_REUSED ·
timeoutMs: รอได้นานสุด (ค่าเริ่มต้น 60 วินาที)
export async function optionpay(method: "GET" | "POST" | "PATCH" | "DELETE",
path: string, payload?: unknown, timeoutMs = 60_000) {
  const body = payload === undefined ? "" : JSON.stringify(payload);
  const timestamp = Math.floor(Date.now() / 1000).toString();
  const signature = createHmac("sha256",
SECRET).update(`${timestamp}.${method}.${path}.${body}`).digest("hex");
  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: { "X-API-Key": API_KEY, "X-Timestamp": timestamp, "X-Signature":
signature, ...(body ? { "Content-Type": "application/json" } : {}) },
    body: body || undefined,
    signal: AbortSignal.timeout(timeoutMs),   // หมดเวลา → throw (ดูหัวข้อ “ส่งสลิป:
timeout และการส่งซ้ำ”)
  });
  return { status: res.status, data: await res.json() };
}
```

```
// ตัวอย่าง: const { data } = await optionpay("POST", "/api/v1/payment/create", {
amount: 100, orderId: "ORD-001" });
```

ผลลัพธ์เมื่อเซ็นไม่ถูกต้อง

กรณี	HTTP	ผลลัพธ์
ไม่ได้ส่ง X-Timestamp / X-Signature	401	`SIGNATURE_REQUIRED: ต้องส่ง X-Timestamp และ X-Signature ทุกคำขอ — ดูวิธีเซ็นในคู่มือ API`
เซ็นด้วย secret ผิด	401	`INVALID_SIGNATURE: X-Signature ไม่ถูกต้อง — เซ็น "timestamp.METHOD.path.body" ด้วย secret ของ key นี้ และส่ง body ให้ตรงกับที่เซ็นทุกตัวอักษร`
timestamp เก่าเกิน 5 นาที	401	`TIMESTAMP_EXPIRED: X-Timestamp ต้องเป็นเวลาปัจจุบัน (unix วินาที) คลาดเคลื่อนไม่เกิน 5 นาที`
ส่ง X-Signature เดิมซ้ำ	401	`SIGNATURE_REUSED: ใช้ X-Signature นี้ไปแล้ว — เซ็นใหม่ทุกคำขอ (timestamp ใหม่)`
API key รูปแบบเดิม (sk_live_)	401	`INVALID_API_KEY: X-API-Key ไม่ถูกต้อง ถูกเพิกถอน หรือร้านค้าไม่ได้เปิดใช้งาน (API key ต้องขึ้นต้นด้วย ak_live_)`

> ส่ง body ให้ตรงกับสตริงที่เซ็นทุกตัวอักษร (อย่าจัดรูปแบบ JSON ใหม่หลังเซ็น) · timestamp คลาดได้ไม่เกิน 5 นาที · ลายเซ็นใช้ได้ครั้งเดียว · ห้ามเซ็นในหน้าเว็บหรือแอปที่ลูกค้าเห็น

การตรวจสลิป

ใช้กับ `POST /payment/slip/verify`, `POST /payment/:ref/slip`, `POST /payment/slip/create` (เมื่อส่ง image) และการแนบสลิปจากหน้าชำระเงินของลูกค้า

ข้อมูลบนสลิป	ต้องตรงกับ	ถ้าไม่ตรง
ยอดเงิน	ยอดของรายการ ทุกสตางค์	INVALID_SLIP
บัญชีผู้รับ + ธนาคาร	บัญชีรับเงินของรายการ	INVALID_SLIP
วันเวลาที่โอน	หลังสร้างรายการ และไม่เก่าเกินกำหนด — ยกเว้นเงินโอนจากรายการก่อนหน้าของลูกค้าคนเดียวกันที่ยังไม่ได้ชำระ (สร้างก่อนเวลาโอน ≤ 24 ชม. บัญชีผู้รับเดียวกัน — ถ้ายังเปิดอยู่ ยกเลิกอัตโนมัติ) → ผ่าน พร้อม carriedFrom	INVALID_SLIP
ชื่อผู้โอน	`metadata.memberName` ตอนสร้างรายการ (ถ้าส่ง)	ไม่ปฏิเสธ — ผู้อื่นโอนแทนได้ · แจ้งใน warnings (และหมายเหตุของรายการ)
สลิปเดียวกัน	ใช้ได้กับรายการเดียว (ตรวจจากไฟล์ และ QR ธนาคาร)	INVALID_SLIP
สลิปตัวอย่างจาก Option Pay	ใช้แนบไม่ได้	`400 SAMPLE_SLIP`

- **มีผู้ให้บริการตรวจสลิป** (EasySlip / SlipOK): ตรวจกับธนาคารจริง — ผ่านครบได้ `PAID` อัตโนมัติ
- **ไม่มีผู้ให้บริการ**: ระบบอ่านข้อมูลจากภาพสลิป — ไม่ตรงได้ `INVALID_SLIP` · ตรงครบได้ `PENDING_REVIEW` รอผู้ดูแลยืนยัน (ไม่อนุมัติอัตโนมัติ)
- ทุกข้อที่ไม่ตรงอยู่ใน `mismatches` (array) พร้อมข้อความบอกสิ่งที่ต้องแก้ · หน้าชำระเงินแนบได้ครั้งเดียว (`409 SLIP_ALREADY_UPLOADED`) เว้นแต่สลิปก่อนหน้าไม่ผ่าน

โครงสร้าง JSON ของผลตรวจสลิป — กรณีปกติ vs เงินโอนจากออเดอร์ก่อนหน้า

ใช่ — โครงสร้างเหมือนกัน · ทุก response ของการตรวจสลิปจาก API (`POST /payment/:ref/slip`, `POST /payment/slip/verify`, `POST /payment/slip/create` ที่ส่ง image) ตอบฟิลด์ (key) ชุดเดียวกันเสมอ ไม่ว่าจะเกิดอะไรขึ้น · ในกรณีปกติ ฟิลด์ที่มีความหมายเฉพาะกรณีพิเศษ **ยังอยู่ครบ แต่เป็นค่าว่าง: `null` หรือ `[]`** — parse ด้วยโค้ดชุดเดียวได้ทุกกรณี

ชื่อฟิลด์ (key)	ชนิดข้อมูล	กรณีปกติ ①	เงินโอนจากออเดอร์ก่อนหน้า ②	มีทั้งสองกรณี
`success`	`boolean`	true	true	✓
`verified`	`boolean`	true	true	✓
`result`	`string`	"PAID"	"PAID"	✓
`message`	`string`	"ตรวจสลิปผ่าน — ออเดอร์ชำระแล้ว (อนุมัติอัตโนมัติ)"	ข้อความเดียวกัน + อธิบายเงินโอนจาก TXN100016 และการยกเลิกอัตโนมัติ	✓
`mismatches`	`string[]`	[]	[]	✓
`warnings`	`string[]`	[]	[]	✓
`carriedFrom`	`string | null`	null	"TXN100016"	✓
`carryOver`	`object | null`	null	{ fromRef, fromOrderId, fromAmount, fromStatus, transferAmount, transferredAt, reason }	✓
`autoCancelled`	`object[]`	[]	[{ ref, orderId, amount, reason }]	✓
`checks`	`object[]`	7 ข้อ ทุกข้อ PASS	7 ข้อ ทุกข้อ PASS (TRANSFER_TIME อธิบายว่าใช้เงินโอนของออเดอร์ก่อนหน้า)	✓
`slip`	`object | null`	{ provider, transRef, amount, transferredAt, sender, receiver }	เหมือนกัน	✓
`nextStep`	`string`	"ไม่ต้องทำอะไรเพิ่ม — ..."	ข้อความเดียวกัน + บันทึกความเชื่อมโยง / ปิด TXN100016 ในระบบร้าน	✓
`transaction`	`object`	{ ... status: "PAID" }	{ ... status: "PAID" }	✓

- **ตัดสินใจด้วย `result`** : `PAID` ส่งสินค้าได้ · `PENDING_REVIEW` รอ webhook · `INVALID_SLIP` แจ้งลูกค้าตาม `mismatches` — ทุกกรณีตอบ HTTP 200

- **อาจเป็น null**: `carriedFrom` `carryOver` `slip` · **อาจเป็น []**: `mismatches` `warnings` `autoCancelled` · `checks` ไม่ว่างเสมอ
- เช็กกรณีพิเศษด้วย `carriedFrom !== null` และ `autoCancelled.length > 0` — ไม่ต้องเช็กว่ามีฟิลด์หรือไม่
- `message` / `detail` / `nextStep` เป็นข้อความภาษาไทยสำหรับแสดงผล อย่า parse
- **ข้อยกเว้น**: HTTP ไม่ใช่ 2xx = ปฏิเสธก่อนตรวจ → `{ error: { code, message, details? } }` · หน้าชำระเงินของลูกค้า (`/payment/checkout/:transactionId/slip`) ตอบแบบย่อ `success verified result message mismatches status`

ตัวอย่างเต็มทั้งสองกรณี — ดู **`POST /PAYMENT/SLIP/VERIFY`** ในหัวข้อ PAYMENT

> ส่ง `metadata.memberName` ให้ตรงกับชื่อบัญชีธนาคารที่ลูกค้าจะใช้โอน (ไม่ต้องมีคำนำหน้า) หรือไม่ต้องส่งถ้าไม่ทราบ

ส่งสลิป: timeout และการส่งซ้ำ

การส่งสลิปใช้เวลาประมาณ **3–5 วินาที** (ส่วนใหญ่คือการตรวจกับธนาคารผ่าน EasySlip) — บางครั้งธนาคารตอบช้าจนนานถึง 20 วินาทีขึ้นไป ระบบของร้านจึงควรรับมือกับ "ไม่ได้รับคำตอบ" ได้ถูกต้อง

> **หมดเวลา (timeout) ไม่ได้แปลว่าส่งไม่สำเร็จ** — เซิร์ฟเวอร์อาจตรวจเสร็จและอนุมัติแล้ว แต่คำตอบมาไม่ถึง · ห้ามเดา: ถามสถานะออเดอร์ก่อนทุกครั้ง · ห้ามสร้างออเดอร์ใหม่เพราะ timeout (จะได้ออเดอร์ซ้ำ)

สิ่งที่เกิดขึ้น	ความหมาย	ระบบของร้านควรทำ
HTTP 200 · `result: PAID`	ตรวจผ่าน ออเดอร์ชำระแล้ว	จบ — ส่งสินค้า/บริการได้
HTTP 200 · `result: PENDING_REVIEW`	รอเจ้าหน้าที่ตรวจ (เช่น ธนาคารไม่ตอบ)	รอ webhook `payment.approved` / `payment.rejected` — ไม่ต้องส่งซ้ำ
HTTP 200 · `result: INVALID_SLIP`	สลิปไม่ผ่าน (ยอด / บัญชี / เวลา / ซ้ำ)	แจ้งลูกค้าตาม `mismatches` — **ส่งสลิปเดิมซ้ำไม่ช่วย** ต้องเป็นสลิปที่ถูกต้อง
หมดเวลา / เครือข่ายขัดข้อง / ไม่มีคำตอบ	ไม่รู้ผล — อาจสำเร็จแล้ว	เรียก `GET /payment/order/:orderId` → `PAID` = จบ · `PENDING` = ส่งสลิปเดิมเข้าออเดอร์เดิมอีกครั้ง
HTTP `502 / 503 / 504`	เซิร์ฟเวอร์ไม่พร้อมชั่วคราว (เช่น ระหว่างอัปเดตระบบ)	รอแล้วส่งซ้ำ (ดูจังหวะด้านล่าง)
HTTP `409 ORDER_NOT_PENDING`	ออเดอร์ไม่รอชำระแล้ว — มักเพราะครั้งก่อนสำเร็จ	เรียก `GET /payment/order/:orderId` ยืนยัน — `PAID` = จบ
HTTP `401 SIGNATURE_REUSED` · `TIMESTAMP_EXPIRED`	ส่งซ้ำด้วยลายเซ็น/เวลาเดิม	เซ็นใหม่ทุกครั้งที่ส่ง (timestamp ใหม่) — helper ด้านล่างทำให้อัตโนมัติ
HTTP `400` อื่น ๆ (`validation_error`, `INVALID_SLIP` ไฟล์ไม่ใช่รูป/ใหญ่เกิน 8 MB, `SAMPLE_SLIP`) · `404 ORDER_NOT_FOUND`	คำขอผิด	**อย่าส่งซ้ำ** — แก้คำขอ (orderId / ไฟล์ / field)

กติกาการส่งซ้ำ

- **ตั้ง timeout 60 วินาที** สำหรับการส่งสลิป (สั้นกว่านี้จะตัดรายการที่ธนาคารตอบช้าทิ้งโดยไม่จำเป็น)
- **ส่งซ้ำได้อย่างปลอดภัย** — สลิปเดิมเข้าออเดอร์เดิมไม่ทำให้ชำระซ้ำ: เงินโอนหนึ่งรายการชำระได้ออเดอร์เดียวเสมอ และออเดอร์ที่ชำระแล้วตอบ `409 ORDER_NOT_PENDING`
- ส่งซ้ำเฉพาะ **timeout / เครือข่ายขัดข้อง / 502 / 503 / 504** — รอ 2, 5, 15 วินาที ไม่เกิน 3 ครั้ง · ถามสถานะออเดอร์ก่อนส่งซ้ำทุกครั้งหลัง timeout
- **เซ็นคำขอใหม่ทุกครั้ง** — ลายเซ็นใช้ได้ครั้งเดียว และ timestamp คลาดได้ไม่เกิน 5 นาที
- **ผลสุดท้ายยึด webhook** `payment.approved` / `payment.rejected` — แม้คำตอบของการส่งสลิปจะหายไป ร้านยังได้รับผลทาง webhook
- ย่อรูปก่อนส่ง (กว้างประมาณ 1000 px, JPEG คุณภาพ 80%) — ส่งเร็วขึ้น และต้องไม่ครอปจน QR หาย
- หน้าชำระเงินของลูกค้า (`/payment/checkout/:transactionId/slip`) แนบได้ครั้งเดียว — ส่งซ้ำหลังสำเร็จได้ `409 SLIP_ALREADY_UPLOADED` ซึ่งแปลว่าได้รับสลิปแล้ว

NEXT.JS — ส่งสลิปพร้อมจัดการ TIMEOUT และส่งซ้ำ

```
// lib/send-slip.ts — ส่งสลิปพร้อมจัดการ timeout / เครือข่ายขัดข้อง (ใช้ optionpay() จาก
lib/optionpay.ts)
import { optionpay } from "./optionpay";

const WAIT_MS = [2_000, 5_000, 15_000];   // รอก่อนส่งซ้ำครั้งที่ 1, 2, 3

const orderStatus = async (orderId: string) =>
  (await optionpay("GET",
`/api/v1/payment/order/${encodeURIComponent(orderId)}`, undefined,
15_000)).data;

export async function sendSlip(orderId: string, imageBase64: string, mime =
"image/jpeg") {
  for (let attempt = 0; ; attempt++) {
    try {
      const r = await optionpay("POST", "/api/v1/payment/slip/verify", {
orderId, image: imageBase64, mime });
      if (r.status === 200) return r.data;                // ตรวจแล้ว: ดู
r.data.result (PAID | PENDING_REVIEW | INVALID_SLIP)
      if (r.status === 409 && r.data.error?.code === "ORDER_NOT_PENDING")
return orderStatus(orderId); // ปิดไปแล้ว — ครั้งก่อนอาจสำเร็จ
      if (![502, 503, 504].includes(r.status)) return r.data; // 4xx อื่น: คำขอผิด
— แก้ อย่าส่งซ้ำ
      // 502 / 503 / 504: เซิร์ฟเวอร์ไม่พร้อมชั่วคราว → ส่งซ้ำด้านล่าง
    } catch {
      // timeout / เครือข่ายขัดข้อง: ไม่รู้ว่าเซิร์ฟเวอร์ตรวจเสร็จหรือยัง → ถามสถานะก่อน อย่าเดา
      const s = await orderStatus(orderId).catch(() => null);
      if (s?.transaction?.status === "PAID") return s;       // ครั้งก่อนสำเร็จแล้ว —
ไม่ต้องส่งซ้ำ
    }
    if (attempt >= WAIT_MS.length) throw new Error("ส่งสลิปไม่สำเร็จ — รอ webhook
payment.approved / payment.rejected หรือแจ้งผู้ดูแล");
    await new Promise((ok) => setTimeout(ok, WAIT_MS[attempt]));   // แล้วส่งสลิป
เดิมเข้าออเดอร์เดิมอีกครั้ง (ปลอดภัย)
```

```
    }
  }
```

รายงานผลการตรวจสลิป (GET /payment/validation-report)

ดูว่าแต่ละรายการชำระเงิน **อนุมัติ** หรือ **ไม่อนุมัติ** เพราะอะไร — ระบบดึงข้อมูลการโอนจริงจากธนาคาร (ผ่านผู้ให้บริการตรวจสลิป) ใหม่ทุกครั้งที่เรียก แล้วแสดงผลการตรวจทุกข้อ พร้อมเหตุผลภาษาไทยและอังกฤษ · รายการที่อนุมัติแสดงก่อน

```
GET https://optionpay.co/api/v1/payment/validation-report?<query>
```

HEADER ที่ต้องส่ง และวิธีได้มา

Header	จำเป็น	ค่า	ได้มาจากไหน
`X-API-Key`	✓	API key (`ak_live_…`)	Merchant Portal → **ตั้งค่า** → การ์ด **Secret key และ API key** → กด **สร้าง secret key** (ครั้งแรก) หรือ **ดู** แล้วคัดลอก
`X-Timestamp`	✓	เวลาปัจจุบัน แบบ unix วินาที	สร้างตอนเรียก เช่น `date +%s` · ต้องคลาดจากเวลาจริงไม่เกิน 5 นาที
`X-Signature`	✓	HMAC-SHA256 แบบ hex ตัวเล็ก	คำนวณจาก secret key (`sk_live_…` หรือ `ss_live_…` ของ key ระดับร้าน) กับข้อความ `<timestamp>.GET.<path+query>.` — เซ็นใหม่ทุกคำขอ
`X-Sub-Merchant`	—	id ร้านย่อย (ตัวเลข)	เฉพาะร้านแม่ที่ต้องการดูรายการของร้านย่อย · ได้ id จาก `GET /sub-merchants` หรือเมนู **ร้านย่อย** ใน Portal · ไม่ต้องรวมในลายเซ็น

QUERY PARAMETERS — ใส่อย่างน้อย 1 ใน REFS / BATCH / DATE / START_TIME (ใส่หลายตัว = ต้องตรงทุกเงื่อนไข)

Parameter	รูปแบบ	ตัวอย่าง (ตามที่ส่งใน URL)	ได้มาจากไหน
`refs`	เลขที่รายการ (TXN...) หรือ orderId คั่นด้วย , (ไม่เกิน 100)	`refs=TXN100001%2CORD-1002`	`transaction.ref` / `transaction.orderId` ที่ได้จาก `POST /payment/create` (orderId = เลขที่ร้านส่งมา หรือ OP-... ที่ระบบสร้างให้) · หรือคอลัมน์ **เลขที่** ในหน้า รายการชำระเงิน · หรือจาก `GET /transactions`
`batch`	ชื่อป้ายกลุ่ม (a-z 0-9 . _ -)	`batch=shop-2026-10-14`	ทีม Option Pay ติดป้ายให้กลุ่มรายการที่ต้องการดูพร้อมกัน แล้วแจ้งชื่อป้ายให้ร้าน
`date`	YYYY-MM-DD (วันที่สร้างรายการเวลาไทย)	`date=2026-10-14`	วันที่ที่ต้องการตรวจ · หรือจาก `transaction.createdAt` (เวลา UTC — แปลงเป็นเวลาไทย +7 ชม.)
`source`	`API` · `SLIP` · `COLLECT` · `PORTAL`	`source=API`	ช่องทางที่สร้างรายการ: API = `/payment/create` · SLIP = `/payment/slip/create` · COLLECT = `/collect` · PORTAL = สร้างใน Portal (ดูได้จาก `transaction.source`)
`start_time`	HH:MM หรือ HH:MM:SS (คู่กับ date) หรือ ISO พร้อมเขตเวลา	`start_time=16%3A52%3A49`	เวลาสร้างรายการเริ่มต้นที่ต้องการ (รวมเวลานั้น)
`end_time`	เหมือน start_time	`end_time=17%3A04%3A10`	เวลาสร้างรายการสิ้นสุด — รวมทั้งวินาที (หรือทั้งนาที ถ้าระบุแค่ HH:MM) นั้น

เข้ารหัสอักขระใน URL ก่อนเซ็น — ลายเซ็นใช้ URL ตามที่ส่งทุกตัวอักษร และเซิร์ฟเวอร์รับ URL แบบเข้ารหัส: `,` → `%2C` · `:` → `%3A` · `+` → `%2B` · ถ้าส่ง `,` หรือ `:` ตรง ๆ จะได้ `401 INVALID_SIGNATURE`

ตัวอย่างที่ใช้งานได้จริง — CURL (ทดสอบกับระบบจริงแล้ว)

```
# 1) ใส่ key ของคุณ (Merchant Portal → ตั้งค่า → Secret key และ API key)
API_KEY="ak_live_YOUR_API_KEY"
```

```
SECRET="sk_live_YOUR_SECRET_KEY"     # key ระดับร้านใช้ ss_live_…

# 2) เลือกรายการ — ตัวอย่าง: 2 รายการ (, ต้องส่งเป็น %2C)
PATHQ="/api/v1/payment/validation-report?refs=TXN100001%2CTXN100002"

# 3) เซ็นคำขอ: HMAC-SHA256(secret, "timestamp.GET.path+query.") — GET ไม่มี body จึงลงท้ายด้วย "."
TS=$(date +%s)
SIG=$(printf '%s' "${TS}.GET.${PATHQ}." | openssl dgst -sha256 -hmac "${SECRET}" | awk '{print $NF}')

# 4) เรียก API — แสดงผลบนจอและบันทึกเป็นไฟล์
curl -sS --max-time 180 "https://optionpay.co${PATHQ}" \
  -H "X-API-Key: ${API_KEY}" \
  -H "X-Timestamp: ${TS}" \
  -H "X-Signature: ${SIG}" \
  | tee validation-report.json
```

เลือกรายการแบบอื่น

```
# ทุกรายการที่สร้างวันที่ 14 ต.ค. 2569 (เวลาไทย) ผ่าน /payment/create
PATHQ="/api/v1/payment/validation-report?date=2026-10-14&source=API"

# เฉพาะช่วงเวลา 16:52:49–17:04:10 ของวันนั้น (: ต้องส่งเป็น %3A)
PATHQ="/api/v1/payment/validation-report?date=2026-10-14&start_time=16%3A52%3A49&end_time=17%3A04%3A10"

# กลุ่มรายการที่ Option Pay ติดป้ายไว้ให้ (batch)
PATHQ="/api/v1/payment/validation-report?batch=shop-2026-10-14"

# ร้านแม่ดูรายการของร้านย่อย id 123 — เพิ่ม header นี้ (ไม่ต้องเซ็น header นี้)
  -H "X-Sub-Merchant: 123"
```

NEXT.JS (ROUTE HANDLER)

```
// app/api/validation-report/route.ts — ใช้ optionpay() จาก lib/optionpay.ts (ดูหัวข้อ “การเซ็นคำขอ”)
import { optionpay } from "@/lib/optionpay";

export async function GET() {
  const refs = ["TXN100001", "TXN100002"].join("%2C");           // , → %2C
  const { status, data } = await optionpay("GET", `/api/v1/payment/validation-report?refs=${refs}`, undefined, 180_000);
  if (status !== 200) return Response.json(data, { status });     // { error: { code, message, details } }
  for (const o of data.orders) console.log(o.ref, o.decision, o.reason_th);
  return Response.json(data);
}
```

ผลลัพธ์จริง — HTTP 200 (ข้อมูลร้าน ลูกค้า และเลขบัญชีเปลี่ยนเป็นตัวอย่าง)

```
{
  "success": true,
  "report": "payment validation — ร้านตัวอย่าง",
  "generatedAt": "2026-10-15T11:37:29+07:00",
  "summary": {
    "total": 2,
    "approved": 1,
    "notApproved": 1,
    "notApprovedWithSlip": 1,
    "noSlipSent": 0
  },
  "howVerified": "Each slip's QR code was sent to EasySlip, which returns the bank's own record of the transfer (sender, receiver, amount, time, reference) — fetched again for this report. The system then checks: the amount matches the order exactly, the receiving account is this order's account, the transfer was made after the order was created, the sender name (warning only), and that the transfer was not used for another order.",
  "orders": [
    {
      "ref": "TXN100001",
      "orderId": "ORD-1001",
      "customer": "สมชาย ใจดี",
      "decision": "APPROVED",
      "currentStatus": "PAID",
      "amounts": {
        "orderAmount": 500,
        "fee": 8,
        "netAmount": 492
      },
      "receivingAccount": "SCB 1234567890 (บริษัท ร้านตัวอย่าง จำกัด)",
      "timeline": {
        "orderCreated": "2026-10-14T16:52:49+07:00",
        "slipUploaded": "2026-10-14T17:42:59+07:00",
        "approvedAt": "2026-10-14T17:42:59+07:00",
        "approvedBy": "system"
      },
      "bankVerification": {
        "source": "EasySlip (bank) — slip image",
        "verified": true,
        "bankReference": "016287100001ABC00001",
        "transferredAt": "2026-10-14T16:53:05+07:00",
        "amount": 500,
        "sender": {
          "name": "นาย สมชาย ใ",
          "bank": "KBANK",
          "account": "xxx-x-x1234-x"
        },
        "receiver": {
          "name": "บริษัท ร้านตัวอย่าง จ",
          "bank": "SCB",
          "account": "xxx-x-x4567-x"
```

```
        }
      },
      "checks": [
        {
          "check": "BANK_VERIFICATION",
          "status": "PASS",
          "detail": "ธนาคารยืนยันว่ามีการโอนจริง · เลขอ้างอิง 016287100001ABC00001"
        },
        {
          "check": "AMOUNT",
          "status": "PASS",
          "detail": "ยอดในสลิป ฿500.00 ตรงกับยอดออเดอร์"
        },
        {
          "check": "TRANSFER_TIME",
          "status": "PASS",
          "detail": "โอนเมื่อ 14 ต.ค. 2569 16:53 — หลังสร้างออเดอร์ (14 ต.ค. 2569
16:52)"
        },
        {
          "check": "RECEIVER_ACCOUNT",
          "status": "PASS",
          "detail": "โอนเข้า SCB xxx-x-x4567-x — ตรงกับบัญชีของออเดอร์ (SCB
1234567890 (บริษัท ร้านตัวอย่าง จำกัด))"
        },
        {
          "check": "SENDER_NAME",
          "status": "PASS",
          "detail": "ผู้โอน นาย สมชาย ใ ตรงกับชื่อลูกค้า"
        },
        {
          "check": "DUPLICATE",
          "status": "PASS",
          "detail": "เงินโอนนี้ไม่ได้ใช้ชำระรายการอื่น"
        }
      ],
      "reason": "The bank confirmed a real transfer of ฿500.00 (ref
016287100001ABC00001) from นาย สมชาย ใ (KBANK xxx-x-x1234-x) to บริษัท ร้านตัวอย่าง จ
(SCB xxx-x-x4567-x) — this order's account — at 2026-10-14T16:53:05+07:00,
after the order was created. The amount matches the order exactly and the
transfer was not used for any other order. All checks passed, so it was
approved.",
      "reason_th": "ธนาคารยืนยันการโอนจริง ฿500.00 (เลขอ้างอิง 016287100001ABC00001)
จาก นาย สมชาย ใ (KBANK xxx-x-x1234-x) ไปยัง บริษัท ร้านตัวอย่าง จ (SCB xxx-x-x4567-x)
ซึ่งเป็นบัญชีของรายการนี้ · โอนหลังสร้างรายการ · ยอดตรงกับรายการพอดี · ไม่เคยใช้กับรายการอื่น ·
ผ่านทุกข้อ จึงอนุมัติ"
    },
    {
      "ref": "TXN100002",
      "orderId": "ORD-1002",
      "customer": "สมหญิง รักดี",
      "decision": "NOT_APPROVED",
```

```
      "currentStatus": "EXPIRED",
      "amounts": {
        "orderAmount": 1500,
        "fee": 24,
        "netAmount": 1476
      },
      "receivingAccount": "SCB 1234567890 (บริษัท ร้านตัวอย่าง จำกัด)",
      "timeline": {
        "orderCreated": "2026-10-14T17:00:37+07:00",
        "slipUploaded": "2026-10-14T18:17:01+07:00",
        "approvedAt": null,
        "approvedBy": null
      },
      "bankVerification": {
        "source": "EasySlip (bank) — slip image",
        "verified": true,
        "bankReference": "016287100002ABC00002",
        "transferredAt": "2026-10-14T17:00:56+07:00",
        "amount": 500,
        "sender": {
          "name": "น.ส. สมหญิง ร",
          "bank": "KBANK",
          "account": "xxx-x-x5678-x"
        },
        "receiver": {
          "name": "บริษัท ร้านตัวอย่าง จ",
          "bank": "SCB",
          "account": "xxx-x-x4567-x"
        }
      },
      "checks": [
        {
          "check": "BANK_VERIFICATION",
          "status": "PASS",
          "detail": "ธนาคารยืนยันว่ามีการโอนจริง · เลขอ้างอิง 016287100002ABC00002"
        },
        {
          "check": "AMOUNT",
          "status": "FAIL",
          "detail": "ยอดในสลิป ฿500.00 แต่ออเดอร์ต้องชำระ ฿1,500.00 (โอนขาด
฿1,000.00)"
        },
        {
          "check": "TRANSFER_TIME",
          "status": "PASS",
          "detail": "โอนเมื่อ 14 ต.ค. 2569 17:00 — หลังสร้างออเดอร์ (14 ต.ค. 2569
17:00)"
        },
        {
          "check": "RECEIVER_ACCOUNT",
          "status": "PASS",
          "detail": "โอนเข้า SCB xxx-x-x4567-x — ตรงกับบัญชีของออเดอร์ (SCB
```

```
1234567890 (บริษัท ร้านตัวอย่าง จำกัด))"
        },
        {
          "check": "SENDER_NAME",
          "status": "PASS",
          "detail": "ผู้โอน น.ส. สมหญิง ร ตรงกับชื่อลูกค้า"
        },
        {
          "check": "DUPLICATE",
          "status": "PASS",
          "detail": "เงินโอนนี้ไม่ได้ใช้ชำระรายการอื่น"
        }
      ],
      "reason": "The bank confirmed a real transfer of ฿500.00 (ref
016287100002ABC00002) to บริษัท ร้านตัวอย่าง จ (SCB xxx-x-x4567-x), but: AMOUNT — ยอด
ในสลิป ฿500.00 แต่ออเดอร์ต้องชำระ ฿1,500.00 (โอนขาด ฿1,000.00). The money is in the
correct account but ฿1000.00 short: create a ฿500.00 order and send the same
slip (within 24 h of the transfer), edit this order to ฿500.00, or collect the
remaining ฿1000.00.",
      "reason_th": "ธนาคารยืนยันการโอนจริง ฿500.00 (เลขอ้างอิง 016287100002ABC00002)
ไปยัง บริษัท ร้านตัวอย่าง จ (SCB xxx-x-x4567-x) แต่ไม่ผ่าน: AMOUNT — ยอดในสลิป ฿500.00 แต่
ออเดอร์ต้องชำระ ฿1,500.00 (โอนขาด ฿1,000.00) · เงินเข้าบัญชีถูกต้องแต่ขาด ฿1000.00 — สร้าง
รายการ ฿500.00 แล้วส่งสลิปเดิม (ภายใน 24 ชม. หลังโอน) หรือแก้ยอดรายการเป็น ฿500.00 หรือให้
ลูกค้าโอนส่วนที่ขาด"
    }
  ]
}
```


คำอธิบายผลลัพธ์

ฟิลด์	ความหมาย
`summary`	จำนวนทั้งหมด · approved · notApproved · notApprovedWithSlip (มีสลิปแต่ไม่ผ่าน) · noSlipSent (ยังไม่ส่งสลิป) · notFound (refs ที่ไม่พบ ถ้ามี)
`orders[].decision`	`APPROVED` = อนุมัติแล้ว · `NOT_APPROVED` = ยังไม่อนุมัติ (ดู reason)
`orders[].currentStatus`	PAID · PENDING · EXPIRED · CANCELLED · REFUNDED
`orders[].amounts`	orderAmount (ยอดรายการ) · fee (ค่าธรรมเนียม) · netAmount (ยอดเข้าร้าน)
`orders[].timeline`	orderCreated · slipUploaded · approvedAt · approvedBy (system = อนุมัติอัตโนมัติ) — เวลาไทย
`orders[].bankVerification`	ข้อมูลการโอนจากธนาคาร: bankReference · transferredAt · amount · sender / receiver (ชื่อ ธนาคาร เลขบัญชีแบบปิดบางส่วน) · null = ยังไม่มีสลิป
`orders[].checks[]`	ผลการตรวจทีละข้อ: BANK_VERIFICATION · AMOUNT · TRANSFER_TIME · RECEIVER_ACCOUNT · SENDER_NAME · DUPLICATE — status PASS · FAIL · WARN · UNKNOWN · SKIPPED พร้อมคำอธิบาย
`orders[].reason / reason_th`	เหตุผลที่อนุมัติ / ไม่อนุมัติ (อังกฤษ / ไทย) สำหรับแสดงผล

ข้อผิดพลาด

HTTP	code	เมื่อไร / แก้อย่างไร
400	`validation_error`	ไม่ได้ระบุ refs / batch / date / start_time หรือรูปแบบผิด — ดู details
400	`TOO_MANY_ORDERS`	เงื่อนไขตรงเกิน 100 รายการ — แบ่งช่วงวันที่/เวลา หรือระบุ refs
401	`INVALID_SIGNATURE`	ลายเซ็นไม่ตรง — ตรวจว่าเซ็น URL แบบเข้ารหัส (%2C %3A) ตรงกับที่ส่ง
401	`TIMESTAMP_EXPIRED` · `SIGNATURE_REUSED`	เวลาเครื่องคลาดเกิน 5 นาที หรือส่งลายเซ็นเดิมซ้ำ — เซ็นใหม่ทุกครั้ง
403	`PERMISSION_DENIED`	ผู้ใช้ไม่มีสิทธิ์ ดูรายการ (view_transactions)
403	`SUB_MERCHANT_NOT_FOUND`	X-Sub-Merchant ไม่ใช่ร้านย่อยของ key นี้

ผลแต่ละครั้งใช้โควตาตรวจสลิป 1 ครั้งต่อรายการที่มีสลิป และใช้เวลาประมาณ 10–20 วินาทีต่อ 40 รายการ — ตั้ง timeout 180 วินาที · สิทธิ์ที่ต้องมี: `view_transactions` (ทุกระดับผู้ใช้มีโดยค่าเริ่มต้น)

สิทธิ์และข้อผิดพลาด

key ของผู้ใช้ร้านค้าใช้ได้เฉพาะสิทธิ์ของระดับผู้ใช้ (รายการสิทธิ์ส่งกลับใน `permissions` ตอน login) · key ที่ผู้ดูแลออกให้ร้านใช้ได้ทุกเส้น · ค่าเริ่มต้น (ผู้ดูแลระบบปรับได้):

สิทธิ์	ความหมาย	เจ้าของร้าน	พนักงาน	ดูอย่างเดียว
`view_dashboard`	ดูภาพรวม ยอดเงิน และข้อมูลร้าน	✓	✓	✓
`view_transactions`	ดูรายการชำระเงิน / payout / คำขอถอน	✓	✓	✓
`create_payment`	สร้างรายการชำระเงิน และแนบ/ตรวจสลิป	✓	✓	—
`cancel_payment`	ยกเลิกรายการชำระเงิน	✓	—	—
`create_payout`	สั่งจ่ายเงินออก (payout)	✓	—	—
`request_withdrawal`	ขอถอนเงิน	✓	—	—
`manage_accounts`	จัดการบัญชีรับเงิน	✓	—	—
`manage_webhooks`	จัดการ webhook	✓	—	—
`manage_sub_merchants`	สร้าง / จัดการร้านย่อย	✓	—	—

HTTP	code	เมื่อไร
400	`validation_error`	ขาด field / รูปแบบผิด (ดู details)
400	`INSUFFICIENT_BALANCE` · `below_min_balance` · `below_min_amount`	ยอดไม่พอ / ต่ำกว่าขั้นต่ำ
400	`INVALID_SLIP` · `SAMPLE_SLIP`	ไฟล์สลิปไม่ใช่รูป/ใหญ่เกิน 8 MB · แนบสลิปตัวอย่าง
401	`INVALID_API_KEY`	API key ผิด ถูกเพิกถอน หรือเป็นรูปแบบเดิม (ต้องเป็น ak_live_)
401	`SIGNATURE_REQUIRED` · `INVALID_SIGNATURE` · `TIMESTAMP_EXPIRED` · `SIGNATURE_REUSED`	ไม่ได้เซ็น · ลายเซ็นไม่ตรง · เวลาคลาดเกิน 5 นาที · ส่งลายเซ็นเดิมซ้ำ
403	`LOGIN_PORTAL_ONLY`	เรียก /auth/login จากภายนอก — ขอ API key ใน Merchant Portal → ตั้งค่า
403	`PERMISSION_DENIED`	ระดับผู้ใช้ไม่มีสิทธิ์ทำสิ่งนี้
404	`ORDER_NOT_FOUND` · `TRANSACTION_NOT_FOUND`	ไม่พบรายการของร้านนี้
409	`DUPLICATE_ORDER_ID` · `ORDER_NOT_PENDING` · `SLIP_ALREADY_UPLOADED`	ซ้ำ / สถานะไม่อนุญาต / หน้าชำระเงินแนบสลิปไปแล้ว
429	`RATE_LIMITED`	login ถี่เกิน 10 ครั้งต่อนาที

ตรวจลายเซ็น callback ของร้านคุณ: ใช้วิธีเซ็นด้วย secret key ของร้าน (`sk_live_…`) — ดูสูตรและโค้ดที่หัวข้อ Callback ที่ระบบส่งถึงร้าน (curl)

Callback ที่ระบบส่งถึงร้าน — รูปแบบคำขอ (curl)

เมื่อสถานะรายการเปลี่ยน Option Pay ยิง `POST` ไปที่ URL webhook ของร้าน (ตั้งที่ `POST /webhooks`) — คำขอจริงเหมือนคำสั่ง curl ด้านล่าง**ทุกตัวอักษร**: header 4 ตัวนี้เท่านั้น (ชื่อตัวพิมพ์เล็ก) และ body เป็น JSON บรรทัดเดียว เรียงพารามิเตอร์ตามตัวอย่าง

Header	ค่า
`content-type`	`application/json`
`user-agent`	`OptionPay-Webhook/1.0`
`x-webhook-event`	ชื่อ event เช่น payment.approved
`x-signature`	`t=<unix วินาที>,v1=<hex>` — เซ็นด้วย **secret key ของร้าน** (`sk_live_…`) 2 ขั้น: ดูกรอบด้านล่าง

Event	ส่งเมื่อ	พารามิเตอร์ใน body (ตามลำดับ)
`payment.approved` · `payment.rejected`	ออเดอร์อนุมัติ / ปฏิเสธ	`event, ref, orderId, amount, status, createdAt, resentAt`
`payout.completed` · `payout.failed`	โอน payout แล้ว / payout ถูกปฏิเสธ	`event, ref, orderId, merchantRef, amount, status, failureReason, createdAt, completedAt`

- `amount` เป็นข้อความทศนิยม 2 ตำแหน่ง · เวลาเป็น ISO 8601 (UTC) · payment: `resentAt` = เวลาที่ส่งครั้งนั้น · payout: `orderId` = `merchantRef` ที่ร้านส่งตอนสร้าง, `failureReason` = เหตุผลเมื่อ failed (ไม่เช่นนั้น `null`)
- **ส่งซ้ำ** (ปลายทางไม่ตอบ 2xx — ลองใหม่สูงสุด 5 ครั้ง หรือเจ้าหน้าที่กดส่งซ้ำ) ใช้คำขอรูปแบบเดิม แต่ `t` และลายเซ็นใหม่ทุกครั้ง — ร้านควรรองรับ event เดิมซ้ำ (idempotent) และปฏิเสธ `t` ที่เก่าเกิน 5 นาที
- `payment.expired` / `payment.refunded` ใช้ body แบบ payment · `withdrawal.*` ใช้ body แบบ payout — ส่งเฉพาะ event ที่ร้านสมัครและเปิดไว้
- ตอบกลับด้วย HTTP 2xx = รับแล้ว · Option Pay บันทึกทุกการส่งพร้อมคำตอบของร้าน และแจ้งในกลุ่ม Telegram ที่ตกลงกับร้าน (ถ้ามี) **พร้อมคำสั่ง curl ที่ส่งจริง** และคำตอบของร้าน — ใช้รันซ้ำเพื่อตรวจสอบได้

สูตรและโค้ดด้านล่างใช้ path ตัวอย่าง `/optionpay/callback` — แทนด้วย path ของ webhook URL ของร้านคุณ (ไม่รวมโดเมนและ query) ทุกตัวอักษร

ลายเซ็น callback ของร้านคุณ**เซ็นด้วย secret key ของร้าน** (`sk_live_…` ตัวเดียวกับที่ใช้เซ็นคำขอ API) ไม่ใช้ webhook secret `whsec_…` — คำนวณ 2 ขั้นต่อ callback (ค่า `key` เปลี่ยนทุกครั้งตาม `t` และ body) · ถ้าสร้าง secret key ใหม่ ต้องใช้ตัวใหม่ตรวจลายเซ็นทันที

ทำไมลายเซ็นที่ร้านคำนวณจะตรงกับของ Option Pay: HMAC-SHA256 ให้ผลเดิมทุกครั้งเมื่อ input เหมือนกัน — และทั้งสองฝั่งใช้ input ชุดเดียวกันครบทุกตัว: (1) **secret key เดียวกัน** — `sk_live_…` ของร้านคุณ · (2) **body เดียวกันทุกตัวอักษร** — ร้านใช้ body ดิบที่ได้รับ ไม่แปลง JSON ก่อน · (3) `t` **เดียวกัน** — อ่านจาก header `x-signature` · (4) **path และสูตรเดียวกัน** — `/optionpay/callback` และ 2 ขั้นด้านล่าง → จึงได้ `v1` เท่ากับที่ Option Pay ส่งมา · ถ้าต่างแม้แต่ตัวเดียว แปลว่า body ถูกแก้ระหว่างทาง ใช้ key ไม่ตรงกัน หรือคำขอไม่ได้มาจาก Option Pay — ให้ปฏิเสธ (401)

อธิบายละเอียด: ทำไมลายเซ็นที่ร้านคำนวณเอง จึงตรงกับลายเซ็นที่ Option Pay ส่งมา

1. HMAC-SHA256 คืออะไร

HMAC-SHA256 คือสูตรคำนวณที่รับของ 2 อย่าง คือ **กุญแจลับ (key)** กับ **ข้อความ (message)** แล้วให้ผลลัพธ์เป็นตัวอักษร 64 ตัว (เลขฐานสิบหก) มีคุณสมบัติสำคัญ 3 ข้อ:

- **ใส่ input เหมือนเดิม ได้ผลลัพธ์เหมือนเดิมทุกครั้ง** — ไม่ว่าจะคำนวณที่ไหน ด้วยภาษาโปรแกรมอะไร (Node.js, PHP, openssl…) ผลก็เท่ากันทุกตัวอักษร
- **input ต่างแม้แต่ 1 ตัวอักษร ผลลัพธ์จะต่างไปทั้งหมด** — ไม่ได้ต่างแค่นิดเดียว จึงเห็นได้ทันทีว่ามีอะไรถูกแก้
- **ย้อนกลับไม่ได้** — เห็นผลลัพธ์แล้วก็หากุญแจลับไม่ได้ และถ้าไม่มีกุญแจลับ ก็สร้างลายเซ็นที่ถูกต้องไม่ได้

2. ทั้งสองฝั่งใช้ input ชุดเดียวกันครบทั้ง 4 ตัว

ลายเซ็นของ callback คำนวณจาก input 4 ตัว ทั้ง Option Pay (ผู้ส่ง) และร้าน (ผู้รับ) มี input ชุดเดียวกันครบทุกตัว:

Input	ฝั่ง Option Pay (ตอนส่ง)	ฝั่งร้าน (ตอนรับ)
1. secret key	`sk_live_…` ของร้านคุณ (เก็บไว้แบบเข้ารหัส)	`sk_live_…` ตัวเดียวกัน — ร้านได้จาก Merchant Portal → ตั้งค่า
2. body	ข้อความ JSON ที่ส่งไป	ข้อความ JSON ที่ได้รับ — ต้องใช้แบบดิบ ทุกตัวอักษรเหมือนที่ได้รับ
3. t (เวลา)	เวลาตอนส่ง (unix วินาที) — ใส่ไว้ใน header x-signature	อ่านจาก header `x-signature: t=…` ที่ได้รับ
4. path + สูตร	`/optionpay/callback` + สูตร 2 ขั้น	`/optionpay/callback` + สูตร 2 ขั้น เดียวกัน (ในคู่มือนี้)

secret key **ไม่เคยถูกส่งไปกับ callback** — ทั้งสองฝั่งมีอยู่แล้วล่วงหน้า สิ่งที่ส่งไปมีแค่ body กับ `t` และ `v1` ใน header ดังนั้นคนกลางที่ดักเห็น callback ก็ปลอมลายเซ็นไม่ได้

3. ขั้นตอนของแต่ละฝั่ง

	Option Pay (ผู้ส่ง)	ร้าน (ผู้รับ)
ขั้นที่ 1	สร้าง body และเลือก `t` = เวลาปัจจุบัน	รับ body ดิบ และอ่าน `t`, `v1` จาก header
ขั้นที่ 2	`key = HMAC-SHA256(sk_live, t + ".POST./optionpay/callback." + body)`	คำนวณ key ด้วยสูตรเดียวกัน
ขั้นที่ 3	`v1 = HMAC-SHA256(key, t + "." + body)`	คำนวณ v1 ด้วยสูตรเดียวกัน → ได้ **v1 ของร้าน**
ขั้นที่ 4	ส่ง `x-signature: t=<t>,v1=<v1>` พร้อม body	เทียบ **v1 ของร้าน** กับ **v1 ใน header** — ตรงกัน = เป็นของจริงจาก Option Pay และ body ไม่ถูกแก้

4. ตัวอย่างตัวเลขจริง (ใช้ key ทดสอบ — ไม่ใช่ key จริงของร้าน)

ค่าต่อไปนี้คำนวณจริงด้วย openssl — ร้านลองคำนวณซ้ำด้วยโค้ดของร้านได้ ต้องได้ค่าเดียวกันทุกตัวอักษร:

รายการ	ค่า
secret key (ทดสอบ)	`sk_live_TESTONLY_not_a_real_key_1234567890`
t	`1792236293`
path	`/optionpay/callback`
body	`{"event":"payment.approved","ref":"TXN100001","orderId":"PD-1","amount":"1000.00","status":"approved","createdAt":"2026-10-17T09:21:20.000Z","resentAt":"2026-10-17T09:24:53.515Z"}`
ผลขั้นที่ 2: key	`1b73526bbe51c46d83195993cf246f011c3aee6328e2a180685a213899c2f5c9`
ผลขั้นที่ 3: v1	`f472d7269a411dec164a6b01b57d60caf3d50361f985ddeca0b3b70a3b70b164`

```
SECRET='sk_live_TESTONLY_not_a_real_key_1234567890'
T='1792236293'
BODY='{"event":"payment.approved","ref":"TXN100001","orderId":"PD-1","amount":"1000.00","status":"approved","createdAt":"2026-10-17T09:21:20.000Z","resentAt":"2026-10-17T09:24:53.515Z"}'
KEY=$(printf '%s' "$T.POST./optionpay/callback.$BODY" | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
```

```
V1=$(printf '%s' "$T.$BODY" | openssl dgst -sha256 -hmac "$KEY" | awk
'{print $NF}')
echo $KEY   #
1b73526bbe51c46d83195993cf246f011c3aee6328e2a180685a213899c2f5c9
echo $V1    #
f472d7269a411dec164a6b01b57d60caf3d50361f985ddeca0b3b70a3b70b164
```

ถ้าเปลี่ยน input แม้แต่นิดเดียว v1 จะต่างไปทั้งหมด:

เปลี่ยนอะไร	v1 ที่ได้
ไม่เปลี่ยน (ถูกต้อง)	`f472d7269a411dec164a6b01b57d60caf3d50361f985ddeca0b3b70a3b70b164`
แก้ยอดใน body จาก `1000.00` เป็น `1000.01`	`ef15008c0bcf3819136087b2c65fec983f260727dc3cd9014fa4c9d65577b448`
เปลี่ยน secret key ตัวสุดท้าย 0 → 1	`ee8a0889edd57bb3d2361372bbc66aa8939774af08db6209d6cd9d3bab5093f6`

5. ถ้าลายเซ็นไม่ตรง — ตรวจทีละข้อ

สาเหตุ	วิธีแก้
ใช้ body ที่แปลงแล้ว (เช่น JSON.parse แล้ว JSON.stringify กลับ, จัดรูปแบบ, เว้นวรรค)	ใช้ body ดิบที่ได้รับ เช่น `await req.text()` ก่อนแปลงเป็น JSON
ใช้ key ผิดตัว	ต้องเป็น `sk_live_…` ของร้านคุณ (ไม่ใช่ API key `ak_live_…` และไม่ใช่ `whsec_…`) · ถ้าเพิ่งสร้าง secret key ใหม่ ต้องใช้ตัวใหม่
path ไม่ตรง	ใช้ `/optionpay/callback` ตรงทุกตัวอักษร ไม่ใส่โดเมน ไม่ใส่ query ไม่มี / ท้าย
ลืมจุด (.) หรือคำว่า POST	`t + ".POST." + path + "." + body` และ `t + "." + body`
ใช้ `t` ผิด	ใช้ `t` จาก header ของ callback นั้น ไม่ใช่เวลาปัจจุบันของเครื่องร้าน
ตัวอักษรภาษาไทยใน body	คำนวณจากข้อความแบบ UTF-8 (ค่าเริ่มต้นของ Node.js / openssl)

สรุป: ลายเซ็นของร้านตรงกับของ Option Pay เพราะ **ใช้ secret key เดียวกัน + body เดียวกันทุกตัวอักษร + t เดียวกัน + path และสูตรเดียวกัน** — HMAC ให้ผลเดิมเสมอเมื่อ input เดิม จึงได้ v1 เท่ากัน · ถ้าไม่ตรง ให้ตอบ 401 และไม่ทำรายการ

ขั้น	สูตร
1. key	`key = HMAC-SHA256(sk_live_…, t + ".POST./optionpay/callback." + body)`
2. v1	`v1 = HMAC-SHA256(key, t + "." + body)`
header	`x-signature: t=<t>,v1=<v1>`

คำนวณ V1 ด้วย OPENSSL

```
KEY=$(printf '%s' "$T.POST./optionpay/callback.$BODY" | openssl dgst -sha256 -
hmac "$SECRET" | awk '{print $NF}')   # SECRET = sk_live_… ของร้าน
V1=$(printf '%s' "$T.$BODY" | openssl dgst -sha256 -hmac "$KEY" | awk '{print
$NF}')
# เทียบกับ v1 ใน header x-signature: t=$T,v1=...
```

โค้ด NEXT.JS (APP ROUTER) — รับ CALLBACK และตรวจลายเซ็น

วางเป็นไฟล์ `app/optionpay/callback/route.ts` แล้วตั้ง `OPTIONPAY_SECRET_KEY` = secret key (`sk_live_…`) ของร้านคุณ ใน `.env` — key ตัวเดียวกับที่ Option Pay ใช้เซ็น callback นี้ · ทดสอบกับลายเซ็นจริงแล้ว: callback จริงได้ 200 · แก้ body / ลายเซ็นเก่าเกิน 5 นาที / ไม่มี header ได้ 401

```
// app/optionpay/callback/route.ts  — Next.js (App Router) · รับ callback จาก
Option Pay และตรวจ x-signature
import { createHmac, timingSafeEqual } from "node:crypto";

export const runtime = "nodejs";        // ใช้ node:crypto
export const dynamic = "force-dynamic";

// secret key (sk_live_…) ของร้านคุณ — ตัวเดียวกับที่ใช้เซ็นคำขอ API และตัวเดียวกับที่ Option
Pay ใช้เซ็น callback นี้
// เก็บใน .env เท่านั้น ห้ามใส่ในโค้ด
const SECRET_KEY = process.env.OPTIONPAY_SECRET_KEY ?? "";
// path ของ webhook URL ที่ตั้งไว้กับ Option Pay — ต้องตรงทุกตัวอักษร (ไม่รวมโดเมนและ
query)
const CALLBACK_PATH = "/optionpay/callback";
const MAX_AGE_SECONDS = 300;            // ปฏิเสธลายเซ็นที่เก่ากว่า 5 นาที (กันการส่งซ้ำ)

const hmacHex = (key: string, message: string) => createHmac("sha256",
key).update(message, "utf8").digest("hex");

/** ตรวจลายเซ็น: key = HMAC(sk_live, t.POST.path.body) · v1 = HMAC(key, t.body)
*/
export function verifyOptionPaySignature(rawBody: string, header: string |
```

```
null, secretKey = SECRET_KEY, now = Date.now()): boolean {
  if (!header || !secretKey) return false;
  const parts = Object.fromEntries(header.split(",").map((p) => {
    const i = p.indexOf("=");
    return [p.slice(0, i).trim(), p.slice(i + 1).trim()];
  }));
  const t = parts.t ?? "";
  const v1 = parts.v1 ?? "";
  if (!/^\d+$/.test(t) || !/^[0-9a-f]{64}$/.test(v1)) return false;
  if (Math.abs(now / 1000 - Number(t)) > MAX_AGE_SECONDS) return false;
  const key = hmacHex(secretKey, `${t}.POST.${CALLBACK_PATH}.${rawBody}`);
  const expected = hmacHex(key, `${t}.${rawBody}`);
  return timingSafeEqual(Buffer.from(expected, "hex"), Buffer.from(v1, "hex"));
}

type OptionPayCallback = {
  event: "payment.approved" | "payment.rejected" | "payout.completed" |
"payout.failed" | string;
  ref: string;
  orderId: string | null;
  amount: string;
  status: string;
  merchantRef?: string | null;
  failureReason?: string | null;
};

export async function POST(req: Request) {
  // อ่าน body ดิบก่อนตรวจ — ห้ามใช้ req.json() แล้วแปลงกลับ เพราะตัวอักษรจะไม่ตรงกับที่เซ็น
  const rawBody = await req.text();
  if (!verifyOptionPaySignature(rawBody, req.headers.get("x-signature"))) {
    return Response.json({ code: 401, message: "invalid signature" }, { status:
401 });
  }
  const cb = JSON.parse(rawBody) as OptionPayCallback;
  switch (cb.event) {
    case "payment.approved": // ออเดอร์ cb.orderId ชำระแล้ว — อัปเดตแบบ idempotent
(ได้ event เดิมซ้ำได้)
      break;
    case "payment.rejected": // ออเดอร์ cb.orderId ไม่สำเร็จ / ยกเลิก
      break;
    case "payout.completed": // payout cb.merchantRef โอนสำเร็จ
      break;
    case "payout.failed":    // payout cb.merchantRef ไม่สำเร็จ —
cb.failureReason
      break;
  }
  return Response.json({ code: 200, message: "success" });
}
```

ตรวจลายเซ็น CALLBACK (NODE.JS)

```
import { createHmac, timingSafeEqual } from "node:crypto";
function verifyCallback(rawBody, header, skLive) {     // header = x-signature:
t=...,v1=...
  const { t, v1 } = Object.fromEntries(header.split(",").map((p) =>
p.split("=")));
  if (Math.abs(Date.now() / 1000 - Number(t)) > 300) return false;
  const key = createHmac("sha256",
skLive).update(`${t}.POST./optionpay/callback.${rawBody}`).digest("hex");
  const want = createHmac("sha256",
key).update(`${t}.${rawBody}`).digest("hex");
  return want.length === v1.length && timingSafeEqual(Buffer.from(want),
Buffer.from(v1));
}
```

PAYMENT.APPROVED — ออเดอร์ได้รับการอนุมัติ — ตรวจสลิปผ่านอัตโนมัติ หรือเจ้าหน้าที่อนุมัติ

```
curl -X POST "https://your-site.com/optionpay/callback" \
  -H "content-type: application/json" \
  -H "user-agent: OptionPay-Webhook/1.0" \
  -H "x-webhook-event: payment.approved" \
  -H "x-signature:
t=1792236293,v1=f64b09e38ec11b730e8cafe131b4a073cfbf082fca0b804b6269f3c7bcdefac
3" \
  --data-raw '{"event":"payment.approved","ref":"TXN100001","orderId":"ORD-
1001","amount":"1000.00","status":"approved","createdAt":"2026-10-
17T09:21:20.000Z","resentAt":"2026-10-17T09:24:53.515Z"}'
```

PAYMENT.REJECTED — ออเดอร์ถูกปฏิเสธ / ยกเลิก — เจ้าหน้าที่ปฏิเสธ ร้านยกเลิกผ่าน API หรือระบบยกเลิกอัตโนมัติ

```
curl -X POST "https://your-site.com/optionpay/callback" \
  -H "content-type: application/json" \
  -H "user-agent: OptionPay-Webhook/1.0" \
  -H "x-webhook-event: payment.rejected" \
  -H "x-signature:
t=1792235865,v1=b1e1ab2c24e954c057868875720c66334a81d5f2d0b9d2ac4e49482966f1a46
8" \
  --data-raw '{"event":"payment.rejected","ref":"TXN100002","orderId":"ORD-
1002","amount":"1400.00","status":"rejected","createdAt":"2026-10-
17T08:49:20.511Z","resentAt":"2026-10-17T11:17:45.470Z"}'
```

PAYOUT.COMPLETED — ทีม OPTION PAY โอนเงิน PAYOUT ให้ผู้รับแล้ว

```
curl -X POST "https://your-site.com/optionpay/callback" \
  -H "content-type: application/json" \
  -H "user-agent: OptionPay-Webhook/1.0" \
  -H "x-webhook-event: payout.completed" \
  -H "x-signature:
t=1792236290,v1=f7b235f7bdd6bc423fa3511de99369165469526fae5bee21e79578a6a9279fb
6" \
  --data-raw '{"event":"payout.completed","ref":"OUT10155","orderId":"WD-
```

```
1001","merchantRef":"WD-
1001","amount":"2000.00","status":"completed","failureReason":null,"createdAt":
"2026-10-17T09:35:04.306Z","completedAt":"2026-10-17T09:45:06.509Z"}'
```

PAYOUT.FAILED — PAYOUT ถูกปฏิเสธ — คืนยอด AMOUNT + FEE เข้ายอดคงเหลือของร้านแล้ว

```
curl -X POST "https://your-site.com/optionpay/callback" \
  -H "content-type: application/json" \
  -H "user-agent: OptionPay-Webhook/1.0" \
  -H "x-webhook-event: payout.failed" \
  -H "x-signature:
t=1792234817,v1=c110fcdb3533f672122bc8ac736824ae12f0257e1cce6b5f13fc47b18fe9ee2
5" \
  --data-raw '{"event":"payout.failed","ref":"OUT10156","orderId":"WD-
1002","merchantRef":"WD-
1002","amount":"302.00","status":"failed","failureReason":"ยกเลิกโดยผู้ดูแล
ระบบ","createdAt":"2026-10-17T06:47:40.956Z","completedAt":"2026-10-
17T07:40:30.505Z"}'
```

ตัวอย่างใช้ค่าสมมติ (URL ร้าน, เลขรายการ) แต่**ลายเซ็นคำนวณจริงด้วยวิธีเดียวกับของจริง** — secret key ทดสอบ `sk_live_demo_for_docs_only` และ path `/optionpay/callback` — ใช้ทดสอบโค้ดตรวจลายเซ็นของร้านได้ทันที (ตอนทดสอบให้ข้ามการเช็คเวลา 5 นาทีชั่วคราว เพราะ `t` ในตัวอย่างเป็นเวลาในอดีต) · ของจริงใช้ `sk_live_…` ของร้านคุณ และ path ของ webhook URL ของร้านคุณ

การยืนยันตัวตน (Authentication)

ทุก request ใต้ `/api/v1/` ต้องส่ง `X-API-Key` + `X-Timestamp` + `X-Signature` (ดูหัวข้อ “การเซ็นคำขอ”) ยกเว้นหน้า checkout / QR / แนบสลิปของลูกค้า · `/auth/login` ใช้ได้เฉพาะภายใน Merchant Portal

POST /auth/login

ออก API Key ใหม่ด้วย secret key — ใช้ได้เฉพาะภายใน Merchant Portal

ไม่ต้องใช้สิทธิ์

AUTH: เฉพาะ Merchant Portal (เซิร์ฟเวอร์ของระบบ) — เรียกจากภายนอกได้ `403 LOGIN_PORTAL_ONLY`

คำอธิบายฟิลด์

```
{
  "secretKey": "ms_live_<secret-key>"   // Merchant Portal ส่งให้เอง
}
```

ตัวอย่าง — CURL

```
# /auth/login ใช้ได้เฉพาะภายใน Merchant Portal — เรียกจากภายนอกได้ 403 เสมอ (ผลจริงด้านล่าง)
curl -X POST "https://optionpay.co/api/v1/auth/login" \
  -H "Content-Type: application/json" \
  --data-raw '{"secretKey":"sk_live_DEMO0000docsonly00000000000000000000000"}'
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// /auth/login ใช้ได้เฉพาะภายใน Merchant Portal — เรียกจากระบบของร้านได้ 403 LOGIN_PORTAL_ONLY เสมอ
// รับ API key ที่ Merchant Portal → ตั้งค่า → Secret key และ API key แทน
export async function POST() {
  const res = await fetch("https://optionpay.co/api/v1/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({"secretKey":"sk_live_DEMO0000docsonly0000000000000000000000"}),
  });
  return Response.json(await res.json(), { status: res.status }); // → 403 (ผลลัพธ์ด้านล่าง)
}
```

ผลลัพธ์ — HTTP 403

```
{
  "error": {
    "code": "LOGIN_PORTAL_ONLY",
    "message": "เรียก /auth/login จากภายนอกไม่ได้ — รับ API key ได้ที่ Merchant
```

```
Portal → ตั้งค่า → Secret key และ API key (สร้าง secret key แล้วได้ API key ทันที)"
  }
}
```

ระบบของร้านไม่ต้องเรียกเส้นนี้ — รับ API key ที่ Merchant Portal → ตั้งค่า → **Secret key และ API key**: กด **สร้าง secret key** ได้ API key ทันที (แสดงในหน้าตั้งค่า) · กด **สร้างใหม่** เมื่อต้องการเปลี่ยน key — ได้ทั้ง secret key และ API key ใหม่ อันเดิมใช้ไม่ได้ทันที · กด **ดู** + ยืนยันชื่อผู้ใช้/รหัสผ่าน เพื่อแสดง key แบบเต็ม · **สิทธิ์ของ API key = ระดับของผู้ใช้** (owner / staff / viewer — กำหนดในหน้าตั้งค่าระบบ) เรียกเส้นที่ไม่มีสิทธิ์ได้ `403 PERMISSION_DENIED` · key ที่ผู้ดูแลออกให้ร้านใช้ได้ทุกเส้น · ทุกการออก/ดู key มี audit log

Payment

สร้าง Payment + QR สำหรับลูกค้า scan จ่าย — ระบบตรวจเงินเข้าผ่าน slip/SMS/Telegram แล้วยิง webhook กลับ · สถานะ: `PENDING | PAID | EXPIRED | CANCELLED | REFUNDED`

POST /payment/create

สร้าง Payment (QR PromptPay)

สิทธิ์: **create_payment** · สร้างรายการชำระเงิน และแนบ/ตรวจสลิป

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

คำอธิบายฟิลด์

```
{
  "amount": 100.00,           // required, > 0 (ทศนิยมไม่เกิน 2 ตำแหน่ง)
  "orderId": "ORD-001",       // optional — เลขออเดอร์ของร้าน ต้องไม่ซ้ำภายในร้าน (ซ้ำ = 409 DUPLICATE_ORDER_ID)
                              //            ไม่ส่ง = Option Pay สร้างให้ (OP-…) ไม่ซ้ำแน่นอน → ดูใน transaction.orderId และ orderIdGenerated: true
  "description": "สินค้า A",    // optional
  "metadata": {                                 // optional ทั้ง block
    "senderAccountNumber": "1234567890",        // เลขบัญชีผู้โอน (≥8 หลักถ้าส่ง)
    "senderBankCode": "SCB",                    // ธนาคารผู้โอน
    "memberName": "สมชาย ใจดี",                // ชื่อลูกค้า (ชื่อผู้โอนในสลิปไม่ตรง = แจ้งใน warnings ไม่ปฏิเสธ)
    "memberNameEn": "Mr. A",                    // optional
    "customerId": "CUST-9"                      // id ลูกค้าฝั่งร้าน, optional
  },
  "bankAccountId": 12         // optional — ต้องเป็นบัญชีของร้าน หรือบัญชีกลาง
}
```

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792505"   # ปกติใช้ TS=$(date +%s)
BODY='{"amount":100,"orderId":"ORD-DOC-001","description":"สินค้า A","metadata":
{"senderAccountNumber":"1234567890","senderBankCode":"SCB","memberName":"สมชาย ใจดี","customerId":"CUST-9"}}'
SIG=$(printf '%s' "$TS.POST./api/v1/payment/create.$BODY" | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 75871079d00b21f2deb12dd833bd4551b5a41d85a9833e942c117f4751deb9cc

curl -X POST "https://optionpay.co/api/v1/payment/create" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
```

```
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
  const path = "/api/v1/payment/create";
  const body = JSON.stringify({
    "amount": 100,
    "orderId": "ORD-DOC-001",
    "description": "สินค้า A",
    "metadata": {
      "senderAccountNumber": "1234567890",
      "senderBankCode": "SCB",
      "memberName": "สมชาย ใจดี",
      "customerId": "CUST-9"
    }
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792505"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                                  // ค่าตัวอย่าง: "75871079d00b21f2deb12dd833bd4551b5a41d85a9833e942c117f4751deb9cc"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792505
สตริงที่เซ็น : 1791792505.POST./api/v1/payment/create.
{"amount":100,"orderId":"ORD-DOC-001","description":"สินค้า A","metadata":
{"senderAccountNumber":"1234567890","senderBankCode":"SCB","memberName":"สม
ชาย ใจดี","customerId":"CUST-9"}}
X-Signature :
75871079d00b21f2deb12dd833bd4551b5a41d85a9833e942c117f4751deb9cc
```

ผลลัพธ์ (HTTP 200) — ตัวอย่าง

```
{
  "success": true,
  "orderIdGenerated": false,   // true = ไม่ได้ส่ง orderId มา Option Pay สร้าง
ให้ (เช่น "OP-MGQ3K7ZB-9F1C2A7D4E") — เก็บ transaction.orderId ไว้
  "transaction": {
    "id": "d4dab18fc0540c9a11654ce9af78a434",   // transaction id — ใช้กับ
checkout / QR / cancel (ความลับของลูกค้า)
    "ref": "TXN100018",               // เลขที่รายการของ Option Pay
    "merchantId": "6",
    "orderId": "ORD-001",             // เลขออเดอร์ของร้าน (ไม่ส่ง = Option Pay
สร้างให้ "OP-…")
    "amount": 350,                    // ยอดที่ลูกค้าต้องโอน — แสดงยอดนี้ให้ลูกค้า
เสมอ
    "fee": 5.25, "netAmount": 344.75, // ค่าธรรมเนียม และยอดสุทธิที่ร้านได้เมื่อชำระ
แล้ว
    "status": "PENDING",              // PENDING | PAID | EXPIRED |
CANCELLED | REFUNDED
    "source": "API",
    "paymentMethod": "BANK_TRANSFER", // หรือ PROMPTPAY ถ้าบัญชีรับเงินมี
PromptPay
    "description": "ข้าวมันไก่ 2 จาน", // null ถ้าไม่ส่ง
    "qrCode": null, "qrCodeUrl": null, // URL รูป QR PromptPay — null ถ้าบัญชี
ไม่มี PromptPay
    "qrExpiry": "2026-10-15T01:37:41.237Z",
    "checkoutUrl": "https://optionpay.co/c?t={id}",
    "createdAt": "2026-10-15T01:22:41.237Z",
    "paidAt": null,
    "copy": { "amount": "350.00" }
  },
  "selectedAccount": { "id": "8", "type": "BANK", "accountNumber":
"1234567890",
                       "accountName": "บริษัท ตัวอย่าง จำกัด", "bankCode":
"SCB", "bankName": "ธนาคารไทยพาณิชย์" }
}

// บัญชีรับเงินที่มี PromptPay: "paymentMethod": "PROMPTPAY", "qrCode":
"https://optionpay.co/api/v1/payment/qr/{id}.png",
// "selectedAccount": { "type": "PROMPTPAY", "accountNumber": "0812345678",
... }
```

คำอธิบายผลลัพธ์ทีละฟิลด์

ฟิลด์	ชนิด	ความหมาย
`success`	boolean	true เมื่อสร้างสำเร็จ (ไม่สำเร็จ = HTTP 4xx พร้อม `{error:{code,message,details}}`)
`orderIdGenerated`	boolean	true = ร้านไม่ได้ส่ง `orderId` มา Option Pay สร้างให้ (`OP-…`) — เก็บ `transaction.orderId` ไว้ใช้อ้างอิง
`transaction.id`	string (32)	transaction id — ใช้กับ `/payment/checkout/:id`, `/payment/qr/:id.png`, `/transactions/:id`, ยกเลิก · ถือเป็นความลับของลูกค้า (อยู่ใน checkoutUrl)
`transaction.ref`	string	เลขที่รายการของ Option Pay (`TXN…`) — ใช้กับ `/payment/:ref/slip` และ `/transactions/:ref`
`transaction.merchantId`	string	รหัสร้านเจ้าของรายการ
`transaction.orderId`	string	เลขออเดอร์ของร้าน (ที่ส่งมา หรือที่ระบบสร้างให้) — ไม่ซ้ำภายในร้าน · ใช้กับ `/payment/order/:orderId` และ `/payment/slip/verify`
`transaction.amount`	number	**ยอดที่ลูกค้าต้องโอน** — เท่ากับที่ร้านส่งมา (เว้นแต่ผู้ดูแลเปิด “ยอดไม่ซ้ำ” ซึ่งอาจเติมสตางค์) · แสดงยอดนี้ให้ลูกค้าเสมอ
`transaction.fee`	number	ค่าธรรมเนียมขาเข้า = amount × % + ค่าคงที่ (ตามที่ตั้งให้ร้าน) ปัดเป็น 2 ตำแหน่ง — คิดเมื่อรายการชำระแล้วเท่านั้น
`transaction.netAmount`	number	amount − fee = ยอดที่เข้ายอดคงเหลือของร้านเมื่อ PAID
`transaction.status`	string	`PENDING` รอชำระ · `PAID` ชำระแล้ว · `EXPIRED` หมดอายุ (ยังแนบสลิปได้) · `CANCELLED` ยกเลิก · `REFUNDED` คืนเงิน
`transaction.source`	string	ช่องทางที่สร้าง: `API` · `PORTAL` · `SLIP` · `COLLECT`
`transaction.paymentMethod`	string	`PROMPTPAY` (บัญชีรับเงินมี PromptPay → มี QR) หรือ `BANK_TRANSFER` (โอนเข้าเลขบัญชี)
`transaction.description`	string | null	รายละเอียดที่ร้านส่งมา
`transaction.qrCode` / `qrCodeUrl`	string | null	URL รูป QR PromptPay (PNG) พร้อมยอด — `null` เมื่อ paymentMethod = BANK_TRANSFER
`transaction.qrExpiry`	ISO datetime | null	เวลาหมดอายุของรายการ (สร้าง + อายุออเดอร์ที่ตั้งไว้ ปกติ 15 นาที) · null = ไม่หมดอายุ
`transaction.checkoutUrl`	string	หน้าชำระเงินสำเร็จรูปของ Option Pay — ส่งให้ลูกค้าเปิดได้เลย (แสดงยอด บัญชี/QR นับถอยหลัง แนบสลิป)
`transaction.createdAt`	ISO datetime	เวลาสร้างรายการ (UTC)
`transaction.paidAt`	ISO datetime | null	เวลาที่ชำระ/อนุมัติ — null จนกว่าจะ PAID
`transaction.copy.amount`	string	ยอดเป็นข้อความ 2 ตำแหน่ง (เช่น "350.00") สำหรับปุ่มคัดลอกยอดในแอป

`selectedAccount.id`	string	รหัสบัญชีรับเงินที่ระบบเลือกให้รายการนี้
`selectedAccount.type`	string	`PROMPTPAY` หรือ `BANK`
`selectedAccount.accountNumber`	string	เบอร์/เลข PromptPay (type = PROMPTPAY) หรือ เลขบัญชีธนาคาร (type = BANK) — ลูกค้าโอนเข้าเลขนี้
`selectedAccount.accountName`	string	ชื่อบัญชีรับเงิน — แสดงให้ลูกค้าตรวจก่อนโอน
`selectedAccount.bankCode / bankName`	string	รหัสและชื่อธนาคารของบัญชีรับเงิน (เช่น SCB · ธนาคารไทยพาณิชย์)

แสดงยอดจาก response เสมอ — ถ้าเปิด “ยอดไม่ซ้ำ” ในตั้งค่าระบบ `amount` จะถูกบวกสตางค์เพื่อจับคู่เงินเข้าอัตโนมัติ (ถ้าลูกค้าโอนยอดที่ร้านส่งมาโดยไม่รวมสตางค์ ระบบยังยอมรับเมื่อธนาคารยืนยันสลิป และปรับ `amount` เป็นยอดที่ได้รับจริง) · บัญชีรับเงิน: บัญชีของร้านก่อน ถ้าไม่มีใช้บัญชีกลางตามการกระจายที่ตั้งไว้ · อายุออเดอร์ตาม “อายุออเดอร์ (นาที)” · **Errors**: orderId ซ้ำ → `409 DUPLICATE_ORDER_ID` · ช่องสตางค์เต็ม → `400 DUPLICATE_PENDING_AMOUNT` · ยอดต่ำกว่ายอดฝากขั้นต่ำ → `400 PAYMENT_CONTRACT_POLICY_VIOLATION` (`details.error: below_min_amount` + `minAmount`) · ไม่มีบัญชีรับเงิน → `400 NO_PAYMENT_ACCOUNT` · ขาด field → `400 validation_error`

POST /payment/slip/create

สร้าง Payment (สลิป) — เปิดออเดอร์รอแนบสลิป

สิทธิ์: **create_payment** · สร้างรายการชำระเงิน และแนบ/ตรวจสลิป

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature` + `Content-Type: application/json`

คำอธิบายฟิลด์

```
{
  "amount": 250.00,           // required
  "orderId": "ORD-002",       // optional, unique ต่อ merchant — ไม่ส่ง = Option Pay สร้างให้ (OP-…)
  "description": "สินค้า B",   // optional
  "image": "<base64>",        // optional — แนบสลิปมาเลย = create + verify ในคำขอเดียว
  "mime": "image/jpeg"        // image/png | image/jpeg | image/webp
}
```

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly0000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly0000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792507"   # ปกติใช้ TS=$(date +%s)
BODY='{"amount":250,"orderId":"ORD-DOC-002","description":"สินค้า B"}'
SIG=$(printf '%s' "$TS.POST./api/v1/payment/slip/create.$BODY" | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 2d8c21fb21c46e135201f698d58fb960bf5f8286489d8f353609bad15df5f538

curl -X POST "https://optionpay.co/api/v1/payment/slip/create" \
```

```
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
  const path = "/api/v1/payment/slip/create";
  const body = JSON.stringify({
    "amount": 250,
    "orderId": "ORD-DOC-002",
    "description": "สินค้า B"
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792507"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                                  // ค่าตัวอย่าง: "2d8c21fb21c46e135201f698d58fb960bf5f8286489d8f353609bad15df5f538"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792507
สตริงที่เซ็น : 1791792507.POST./api/v1/payment/slip/create.
{"amount":250,"orderId":"ORD-DOC-002","description":"สินค้า B"}
```

```
X-Signature :
2d8c21fb21c46e135201f698d58fb960bf5f8286489d8f353609bad15df5f538
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "orderIdGenerated": false,
  "transaction": {
    "id": "70d7c30c515b19110c95775386c308cb",
    "ref": "TXN100012",
    "merchantId": "6",
    "orderId": "ORD-DOC-002",
    "amount": 250,
    "fee": 3.75,
    "netAmount": 246.25,
    "status": "PENDING",
    "source": "SLIP",
    "paymentMethod": "BANK_TRANSFER",
    "description": "สินค้า B",
    "qrCode": null,
    "qrCodeUrl": null,
    "qrExpiry": "2026-10-12T08:23:28.238Z",
    "checkoutUrl": "https://optionpay.co/c?
t=70d7c30c515b19110c95775386c308cb",
    "createdAt": "2026-10-12T08:08:28.238Z",
    "paidAt": null,
    "copy": {
      "amount": "250.00"
    }
  },
  "selectedAccount": {
    "id": "8",
    "type": "BANK",
    "accountNumber": "1234567890",
    "accountName": "บริษัท ตัวอย่าง จำกัด",
    "bankCode": "SCB",
    "bankName": "ธนาคารไทยพาณิชย์"
  }
}
```


ไม่บวกสตางค์ — `amount` ที่ตอบกลับเท่ากับที่ส่งมาเสมอ · ถ้าส่ง `image` มาด้วย response จะเป็นแบบ /payment/slip/verify · Errors เหมือน /payment/create

POST /payment/slip/verify

ยืนยันสลิป (Slip Verification) — ตรวจแล้ว mark PAID

สิทธิ์: **create_payment** · สร้างรายการชำระเงิน และแนบ/ตรวจสลิป

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

คำอธิบายฟิลด์

```
{
  "orderId": "ORD-002",     // required
  "image": "<base64>",      // required, ไม่เกิน 8 MB
  "mime": "image/jpeg"      // required: image/png | image/jpeg |
image/webp
}
```

ตัวอย่าง — CURL

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น
ห้ามส่ง
TS="1791792509"   # ปกติใช้ TS=$(date +%s)
BODY="{\"orderId\":\"ORD-DOC-
002\",\"mime\":\"image/png\",\"image\":\"$(base64 < slip.png | tr -d
'\n')\"}"   # slip.png = รูปสลิป
SIG=$(printf '%s' "$TS.POST./api/v1/payment/slip/verify.$BODY" | openssl
dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = (ขึ้นกับไฟล์รูปที่แนบ — คำนวณด้วยคำสั่งด้านบน)

curl -X POST "https://optionpay.co/api/v1/payment/slip/verify" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น
ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";
import { readFile } from "node:fs/promises";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง:
"ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง:
"sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
  const path = "/api/v1/payment/slip/verify";
  const body = JSON.stringify({
    "orderId": "ORD-DOC-002",
    "mime": "image/png",
    "image": (await readFile("slip.png")).toString("base64")
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง:
```

```
"1791792509"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ขึ้นกับไฟล์รูป
ที่แนบ

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ผลลัพธ์ — ฟิลด์ชุดเดียวกันทุกกรณี (HTTP 200)

```
// ทุกผลลัพธ์ของการตรวจสลิปตอบ HTTP 200 และมีฟิลด์ครบชุดเดียวกันเสมอ (กรณีปกติ / เงินโอน
จากออเดอร์ก่อนหน้า / ไม่ผ่าน / รอตรวจ)
// HTTP ไม่ใช่ 2xx = ถูกปฏิเสธก่อนตรวจ → { "error": { "code", "message",
"details"? } }

// ① กรณีปกติ — ผ่าน อนุมัติอัตโนมัติ
{
  "success": true,                // false เฉพาะ INVALID_SLIP
  "verified": true,               // true เฉพาะ PAID
  "result": "PAID",               // PAID | PENDING_REVIEW | INVALID_SLIP
← ใช้ตัวนี้ตัดสินใจ
  "message": "ตรวจสลิปผ่าน — ออเดอร์ชำระแล้ว (อนุมัติอัตโนมัติ)",
  "mismatches": [],               // string[] — ข้อที่ไม่ผ่าน (มีค่าเมื่อ
INVALID_SLIP)
  "warnings": [],                 // string[] — คำเตือนที่ไม่ทำให้ไม่ผ่าน (เช่น ชื่อ
ผู้โอนไม่ตรง)
  "carriedFrom": null,            // string | null — ref ออเดอร์ก่อนหน้าที่เงิน
โอนนี้มาจาก
  "carryOver": null,              // object | null — รายละเอียดของ
carriedFrom
  "autoCancelled": [],            // object[] — ออเดอร์อื่นที่ถูกยกเลิกอัตโนมัติ
  "checks": [                     // object[] — ไม่ว่างเสมอ · ผลการตรวจทีละข้อ
    { "check": "BANK_VERIFICATION", "status": "PASS", "detail": "ธนาคารยืนยัน
ว่ามีการโอนจริง (easyslip) · เลขอ้างอิง 016287100512BTF04211" },
    { "check": "AMOUNT",           "status": "PASS", "detail": "ยอดในสลิป
฿250.00 ตรงกับยอดออเดอร์" },
    { "check": "TRANSFER_TIME",    "status": "PASS", "detail": "โอนเมื่อ 14
ต.ค. 2569 10:05 — หลังสร้างออเดอร์ (14 ต.ค. 2569 10:01)" },
    { "check": "RECEIVER_ACCOUNT", "status": "PASS", "detail": "โอนเข้า SCB
```

```
xxx-x-x6378-x — ตรงกับบัญชีของออเดอร์ (SCB 1234567890 (บจก. ตัวอย่าง))" },
    { "check": "SENDER_NAME",       "status": "PASS", "detail": "ผู้โอน นาย
สมชาย ใจดี ตรงกับชื่อลูกค้า" },
    { "check": "SLIP_AGE",          "status": "PASS", "detail": "ไม่เก่าเกิน 6
ชั่วโมง" },
    { "check": "DUPLICATE",         "status": "PASS", "detail": "สลิปและเงิน
โอนนี้ยังไม่เคยใช้ชำระรายการอื่น" }
  ],
  "slip": {                        // object | null — ข้อมูลจากธนาคาร (null =
ตรวจกับธนาคารไม่ได้)
    "provider": "easyslip", "transRef": "016287100512BTF04211", "amount":
250,
    "transferredAt": "2026-10-14T03:05:00.000Z",
    "sender":   { "name": "นาย สมชาย ใจดี", "bank": "KBANK", "account":
"xxx-x-x1234-x" },
    "receiver": { "name": "บจก. ตัวอย่าง",  "bank": "SCB",   "account": "xxx-
x-x6378-x" }
  },
  "nextStep": "ไม่ต้องทำอะไรเพิ่ม — ออเดอร์ชำระแล้ว ส่งสินค้า/บริการได้",
  "transaction": { ... "ref": "TXN100017", "orderId": "ORD-002", "status":
"PAID", "paidAt": "2026-10-14T03:05:20.000Z" ... }
}

// ② เงินโอนจากออเดอร์ก่อนหน้า (ฟิลด์เหมือน ① ทุกตัว — ต่างกันแค่ค่า)
//    ลูกค้าเปิด TXN100016 ยอด ฿250 โอนมา ฿200 → ร้านเปิด TXN100017 ยอด ฿200 แล้ว
แนบสลิปเดิม
{
  "success": true, "verified": true, "result": "PAID",
  "message": "ตรวจสลิปผ่าน — ออเดอร์ชำระแล้ว (อนุมัติอัตโนมัติ) · ใช้เงินโอน 200.00 บาท
(โอนเมื่อ 14 ต.ค. 2569 10:05) จากออเดอร์ก่อนหน้า TXN100016 (ORD-001) ยอด 250.00
บาท ที่ยังไม่ได้ชำระ — ลูกค้าคนเดียวกัน บัญชีผู้รับเดียวกัน · ยกเลิก TXN100016 (ORD-001)
อัตโนมัติ — ลูกค้าชำระผ่าน TXN100017 แทน (ใช้เงินโอนเดียวกัน)",
  "mismatches": [],
  "warnings": [],
  "carriedFrom": "TXN100016",
  "carryOver": {
    "fromRef": "TXN100016", "fromOrderId": "ORD-001", "fromAmount": 250,
    "fromStatus": "CANCELLED_AUTOMATICALLY",   // EXPIRED | CANCELLED |
CANCELLED_AUTOMATICALLY (ยังเปิดอยู่ → ถูกปิดตอนนี้)
    "transferAmount": 200, "transferredAt": "2026-10-14T03:05:00.000Z",
    "reason": "เงินโอนนี้เกิดขึ้นหลังสร้างออเดอร์ก่อนหน้าของลูกค้าคนเดียวกัน (บัญชีผู้รับเดียวกัน)
ซึ่งยังไม่ได้ชำระ และยังไม่เคยใช้ชำระออเดอร์ใด จึงใช้ชำระออเดอร์นี้แทน"
  },
  "autoCancelled": [
    { "ref": "TXN100016", "orderId": "ORD-001", "amount": 250, "reason":
"ลูกค้าชำระผ่าน TXN100017 แทน (ใช้เงินโอนเดียวกัน)" }
  ],
  "checks": [
    { "check": "BANK_VERIFICATION", "status": "PASS", "detail": "..." },
    { "check": "AMOUNT",            "status": "PASS", "detail": "ยอดในสลิป
฿200.00 ตรงกับยอดออเดอร์" },
    { "check": "TRANSFER_TIME",     "status": "PASS", "detail": "โอนเมื่อ 14
```

```
ต.ค. 2569 10:05 — ก่อนสร้างออเดอร์ (14 ต.ค. 2569 10:20) — แต่เป็นเงินโอนของออเดอร์
ก่อนหน้า TXN100016 ที่ยังไม่ได้ชำระ (ลูกค้าคนเดียวกัน) จึงใช้แทนได้" },
    { "check": "RECEIVER_ACCOUNT",  "status": "PASS", "detail": "..." },
    { "check": "SENDER_NAME",       "status": "PASS", "detail": "..." },
    { "check": "SLIP_AGE",          "status": "PASS", "detail": "อยู่ในช่วง 24
ชั่วโมงของเงินโอนจากออเดอร์ก่อนหน้า" },
    { "check": "DUPLICATE",         "status": "PASS", "detail": "สลิปและเงิน
โอนนี้ยังไม่เคยใช้ชำระรายการอื่น" }
  ],
  "slip": { ... เหมือน ① ... },
  "nextStep": "ไม่ต้องทำอะไรเพิ่ม — ออเดอร์ชำระแล้ว ส่งสินค้า/บริการได้ · บันทึกว่าออเดอร์นี้
ชำระด้วยเงินโอนของ TXN100016 · อัปเดตออเดอร์ TXN100016 เป็นยกเลิกในระบบของร้าน",
  "transaction": { ... "ref": "TXN100017", "status": "PAID" ... }
}

// ③ สลิปไม่ผ่าน (HTTP 200 เช่นกัน · ออเดอร์ยัง PENDING แนบใหม่ได้)
{
  "success": false, "verified": false, "result": "INVALID_SLIP",
  "message": "ยอดเงินในสลิป ฿200.00 ไม่ตรงกับยอดที่ต้องชำระ ฿250.00 — ต้องโอนตรงยอด
ทุกสตางค์",
  "mismatches": [ "ยอดเงินในสลิป ฿200.00 ไม่ตรงกับยอดที่ต้องชำระ ฿250.00 — ต้องโอน
ตรงยอดทุกสตางค์" ],
  "warnings": [],
  "carriedFrom": null, "carryOver": null, "autoCancelled": [],
  "checks": [
    { "check": "BANK_VERIFICATION", "status": "PASS", "detail": "..." },
    { "check": "AMOUNT",            "status": "FAIL", "detail": "ยอดในสลิป
฿200.00 แต่ออเดอร์ต้องชำระ ฿250.00 (โอนขาด ฿50.00)" },
    ...
  ],
  "slip": { ... },
  "nextStep": "สลิปไม่ผ่าน — แจ้งลูกค้าตามข้อที่ FAIL ใน checks แล้วแนบสลิปที่ถูกต้องใหม่
(ออเดอร์ยังรอชำระ) · ถ้าลูกค้าโอนยอดไม่ตรง สร้างออเดอร์ใหม่ตามยอดที่โอนจริงแล้วแนบสลิปเดิม
ได้",
  "transaction": { ... "status": "PENDING" ... }
}

// ④ PENDING_REVIEW (ตรวจกับธนาคารไม่ได้ / ตั้งให้เจ้าหน้าที่ยืนยัน): success true,
verified false,
//    slip = null ได้, BANK_VERIFICATION = UNKNOWN · รอ webhook
payment.approved / payment.rejected
```

โครงสร้าง JSON: กรณีปกติ vs เงินโอนจากออเดอร์ก่อนหน้า
ใช่ — โครงสร้างเหมือนกัน · ทุก response ของการตรวจสลิปจาก API (`POST /payment/:ref/slip` , `POST /payment/slip/verify` , `POST /payment/slip/create` ที่ส่ง image) ตอบฟิลด์ (key) ชุดเดียวกันเสมอ ไม่ว่าจะเกิดอะไรขึ้น · ในกรณีปกติ ฟิลด์ที่มีความหมายเฉพาะกรณีพิเศษ **ยังอยู่ครบ แต่เป็นค่าว่าง: `null` หรือ `[]`** — parse ด้วยโค้ดชุดเดียวได้ทุกกรณี

ชื่อฟิลด์ (key)	ชนิดข้อมูล	กรณีปกติ ①	เงินโอนจากออเดอร์ก่อนหน้า ②	มีทั้งสองกรณี
`success`	boolean	true	true	✓
`verified`	boolean	true	true	✓
`result`	string	"PAID"	"PAID"	✓
`message`	string	"ตรวจสลิปผ่าน — ออเดอร์ชำระแล้ว (อนุมัติอัตโนมัติ)"	ข้อความเดียวกัน + อธิบายเงินโอนจาก TXN100016 และการยกเลิกอัตโนมัติ	✓
`mismatches`	string[]	[]	[]	✓
`warnings`	string[]	[]	[]	✓
`carriedFrom`	string | null	null	"TXN100016"	✓
`carryOver`	object | null	null	{ fromRef, fromOrderId, fromAmount, fromStatus, transferAmount, transferredAt, reason }	✓
`autoCancelled`	object[]	[]	[{ ref, orderId, amount, reason }]	✓
`checks`	object[]	7 ข้อ ทุกข้อ PASS	7 ข้อ ทุกข้อ PASS (TRANSFER_TIME อธิบายว่าใช้เงินโอนของออเดอร์ก่อนหน้า)	✓
`slip`	object | null	{ provider, transRef, amount, transferredAt, sender, receiver }	เหมือนกัน	✓
`nextStep`	string	"ไม่ต้องทำอะไรเพิ่ม — ..."	ข้อความเดียวกัน + บันทึกความเชื่อมโยง / ปิด TXN100016 ในระบบร้าน	✓
`transaction`	object	{ ... status: "PAID" }	{ ... status: "PAID" }	✓

สำหรับนักพัฒนา — อ่านผลอย่างไร · **HTTP 200 = ตรวจแล้ว** (ทั้งผ่าน ไม่ผ่าน และรอตรวจ) มีฟิลด์ครบชุดเดียวกันทุกกรณี: `success` `verified` `result` `message` `mismatches` `warnings` `carriedFrom` `carryOver` `autoCancelled` `checks` `slip` `nextStep` `transaction` — ไม่มีฟิลด์ที่หายไปหรือเพิ่มขึ้นตามกรณี · **อาจเป็น null**: `carriedFrom` `carryOver` `slip` · **อาจเป็น array ว่าง**: `mismatches` `warnings` `autoCancelled` · `checks` ไม่ว่างเสมอ — แต่ละตัวคือ `{check, status, detail}` , check: BANK_VERIFICATION · AMOUNT · TRANSFER_TIME · RECEIVER_ACCOUNT · SENDER_NAME · IMAGE_READ · SLIP_AGE · DUPLICATE · APPROVAL (อาจเพิ่มในอนาคต — ข้ามตัวที่ไม่รู้จัก) · status: PASS · FAIL · WARN · UNKNOWN · SKIPPED · **ตัดสินใจด้วย `result` ตัวเดียว**: `PAID` → ส่งสินค้าได้ · `PENDING_REVIEW` → รอ webhook · `INVALID_SLIP` → แจ้งลูกค้าตาม `mismatches` · ถ้า `carriedFrom` ไม่ใช่ null ให้บันทึกความเชื่อมโยง และปิดออเดอร์ใน `autoCancelled` ในระบบของร้าน (จะได้ webhook `payment.rejected` ด้วย) · `message` / `detail` / `nextStep` เป็นข้อความภาษาไทยสำหรับแสดงผล อย่า parse · **HTTP ไม่ใช่ 2xx** = ปฏิเสธก่อนตรวจ ใช้รูปแบบ error ปกติ `{error:{code,message,details?}}` · หน้าชำระเงินของลูกค้า (`/payment/checkout/:transactionId/slip`) ตอบแบบย่อ: `success verified result message mismatches status` · **Timeout / เครือข่ายขัดข้อง**: ไม่ได้แปลว่าส่งไม่สำเร็จ — เรียก `GET /payment/order/:orderId` ก่อน (PAID = จบ · PENDING = ส่งสลิปเดิมเข้าออเดอร์เดิมซ้ำได้อย่างปลอดภัย) · ตั้ง timeout 60 วินาที · ส่งซ้ำเฉพาะ timeout / 502 / 503 / 504 (รอ 2, 5, 15 วินาที) และเช็น

ใหม่ทุกครั้ง · `409 ORDER_NOT_PENDING` หลังส่งซ้ำ = ครั้งก่อนสำเร็จแล้ว · ผลสุดท้ายยึด webhook

การตรวจสลิป — (1) **สลิปตัวอย่าง**จาก Option Pay → `400 SAMPLE_SLIP` · (2) **สลิปซ้ำ**: สลิปเดียวกัน (ไฟล์เดียวกัน หรือ QR ธนาคารเดียวกัน) ที่อยู่กับรายการอื่นแล้ว → `INVALID_SLIP` · (3) **เทียบข้อมูลในสลิปกับรายการ**: ยอดเงินตรงทุกสตางค์ · บัญชีผู้รับ + ธนาคารเป็นบัญชีของรายการ · เวลาโอนหลังสร้างรายการและไม่เก่าเกินกำหนด — ทุกข้อที่ไม่ตรงอยู่ใน `mismatches` · **เงินโอนจากรายการก่อนหน้า**: สลิปที่โอนก่อนสร้างรายการ ผ่านได้เมื่อร้านเดียวกันมีรายการที่ยังไม่ได้ชำระ (หมดอายุ / ยกเลิก / ยังเปิดอยู่ — รายการที่ยังเปิดจะถูกยกเลิกอัตโนมัติ + webhook `payment.rejected`) บนบัญชีผู้รับเดียวกัน สร้างก่อนเวลาโอนไม่เกิน 24 ชม. และผู้โอนเป็นลูกค้าคนเดียวกับรายการนั้น (ชื่อ หรือเลขบัญชีผู้โอน) — ผลมี `carriedFrom` = เลขรายการก่อนหน้า, `carryOver` = รายละเอียด (fromRef, fromOrderId, fromAmount, fromStatus: EXPIRED / CANCELLED / CANCELLED_AUTOMATICALLY, transferAmount, transferredAt, reason) และ `message` อธิบายเป็นภาษาไทย · `autoCancelled` = รายการที่ถูกยกเลิกอัตโนมัติพร้อมเหตุผล (ว่าง = ไม่มี) · **ทุกผลลัพธ์ของการแนบสลิปมี** `checks` = ผลการตรวจทีละข้อ (BANK_VERIFICATION, AMOUNT, TRANSFER_TIME, RECEIVER_ACCOUNT, SENDER_NAME, SLIP_AGE, DUPLICATE · status: PASS / FAIL / WARN / UNKNOWN / SKIPPED พร้อมคำอธิบาย), `slip` = ข้อมูลจากธนาคาร (ยอด เวลาโอน เลขอ้างอิง ผู้โอน ผู้รับ — null ถ้าตรวจกับธนาคารไม่ได้) และ `nextStep` = สิ่งที่ร้านควรทำต่อ · **สลิปเดียวกันที่เคยไม่ผ่าน**ในรายการอื่นของร้านที่ยังเปิดอยู่ (เช่น โอน ฿500 ให้รายการ ฿250 แล้วผ่านในรายการใหม่ ฿500): รายการนั้นถูกยกเลิกอัตโนมัติ + webhook `payment.rejected` · **ชื่อผู้โอน**ไม่ตรงกับ `metadata.memberName` (เช่น สามีโอนแทนภรรยา) **ไม่ปฏิเสธ** — แจ้งใน `warnings` และยังอนุมัติได้ · **มีผู้ให้บริการตรวจสลิป** (EasySlip / SlipOK ในตั้งค่าระบบ): ตรวจกับธนาคารจริง ผ่านครบ → `PAID` อัตโนมัติ (เว้นแต่ตั้ง “ต้องให้ผู้ดูแลยืนยันก่อน”) · **ไม่มีผู้ให้บริการ / ผู้ให้บริการไม่ตอบ**: ระบบอ่านข้อมูลจากภาพสลิป (OCR ไทย+อังกฤษ) — ยอด บัญชีผู้รับ หรือเวลาไม่ตรง → `INVALID_SLIP` · ตรงครบ หรืออ่านไม่ได้ → `PENDING_REVIEW` ให้ผู้ดูแลยืนยัน (**ไม่อนุมัติอัตโนมัติ** เพราะภาพแก้ไขได้) · ชื่อผู้โอน/ธนาคารที่ยืนยันจากภาพไม่ได้ จะแจ้งผู้ดูแลแทนการปฏิเสธ · **Errors**: ไฟล์ไม่ใช่รูป/ใหญ่เกิน → `400 INVALID_SLIP` · ออเดอร์ไม่ PENDING → `409 ORDER_NOT_PENDING` · ไม่พบ orderId → `404 ORDER_NOT_FOUND`

POST `/payment/:ref/slip`

แนบสลิปเข้าออเดอร์ตาม ref หรือ transaction id — shape เดิม

สิทธิ์: **create_payment** · สร้างรายการชำระเงิน และแนบ/ตรวจสลิป

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

คำอธิบายฟิลด์

```
{ "image": "<base64>", "mime": "image/png" }
```

ตัวอย่าง — CURL

```
API_KEY="ak_live_DEMO0000docsonly0000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly0000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792511"   # ปกติใช้ TS=$(date +%s)
BODY="{\"mime\":\"image/png\",\"image\":\"$(base64 < slip.png | tr -d '\n')\"}"   # slip.png = รูปสลิป
SIG=$(printf '%s' "$TS.POST./api/v1/payment/TXN100013/slip.$BODY" | openssl
```

```
dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = (ขึ้นกับไฟล์รูปที่แนบ — คำนวณด้วยคำสั่งด้านบน)

curl -X POST "https://optionpay.co/api/v1/payment/TXN100013/slip" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";
import { readFile } from "node:fs/promises";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
  const path = "/api/v1/payment/TXN100013/slip";
  const body = JSON.stringify({
    "mime": "image/png",
    "image": (await readFile("slip.png")).toString("base64")
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792511"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ขึ้นกับไฟล์รูปที่แนบ

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ผลลัพธ์ — ฟิลด์ชุดเดียวกันทุกกรณี (HTTP 200)

```
เหมือน /payment/slip/verify
```

ตรวจสลิปตามกฎเดียวกับ /payment/slip/verify · แนบทับสลิปเดิมได้ (หน้าชำระเงินของลูกค้าแนบได้ครั้งเดียว)

GET /payment/validation-report

รายงานผลการตรวจ (อนุมัติ / ไม่อนุมัติ) พร้อมเหตุผลและข้อมูลจากธนาคาร

ไม่ต้องใช้สิทธิ์ AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

คำอธิบายฟิลด์

```
// Query — อย่างน้อย 1 ใน refs / batch / date / start_time (ใช้ร่วมกันได้ = ต้องตรงทุกเงื่อนไข) · ไม่เกิน 100 รายการต่อครั้ง
refs=TXN100015%2CTXN100030%2CORD-001   // เลขที่ TXN… หรือ orderId คั่นด้วย , — ส่ง , เป็น %2C (ลายเซ็นใช้ URL ตามที่ส่ง)
batch=shop-2026-10-14                  // ป้ายกลุ่มรายการที่ Option Pay ตั้งให้ (เช่น ชุดที่ตรวจข้ำพร้อมกัน)
date=2026-10-14                        // รายการที่สร้างวันนั้น (เวลาไทย)
source=API                             // ช่องทางที่สร้าง: API (/payment/create) · SLIP (/payment/slip/create) · COLLECT · PORTAL
start_time=16%3A52%3A49                // สร้างตั้งแต่ (HH:MM[:SS] คู่กับ date หรือ ISO เช่น 2026-10-14T16%3A52%3A00%2B07%3A00) — ส่ง : เป็น %3A
end_time=17%3A04%3A10                  // สร้างถึง (รวมทั้งวินาที/นาทีนั้น)
```

คู่มือเต็ม พร้อมวิธีได้มาของทุก parameter และตัวอย่างที่ใช้งานได้จริง: Merchant Portal → คู่มือ API → รายงานผลการตรวจสลิป · ข้อมูลธนาคารดึงใหม่จากผู้ให้บริการตรวจสลิปทุกครั้งที่เรียก — **ใช้โควตา 1 ครั้งต่อสลิป** และใช้เวลาประมาณ 10–20 วินาทีต่อ 40 รายการ (ตั้ง timeout 180 วินาที) · เฉพาะรายการของร้านนี้ (ร้านแม่ใช้ `X-Sub-Merchant` ดูร้านย่อยได้) · สิทธิ์ `view_transactions` · Errors: ไม่ระบุ refs/date → `400 validation_error` · เกิน 100 รายการ → `400 TOO_MANY_ORDERS`

GET /payment/order/:orderId

ตรวจสอบ Payment ตาม Order ID

สิทธิ์: **view_transactions** · ดูรายการชำระเงิน / payout / คำขอถอน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792513"   # ปกติใช้ TS=$(date +%s)
```

```
SIG=$(printf '%s' "$TS.GET./api/v1/payment/order/ORD-DOC-001." | openssl
dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 159ab4b58d93e2ac834c33312f43550fbe5942d56324a1f005141a216232b883

curl "https://optionpay.co/api/v1/payment/order/ORD-DOC-001" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น
ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง:
"ak_live_DEMO0000docsonly00000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง:
"sk_live_DEMO0000docsonly00000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/payment/order/ORD-DOC-001";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง:
"1791792513"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง:
"159ab4b58d93e2ac834c33312f43550fbe5942d56324a1f005141a216232b883"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792513
สตริงที่เซ็น : 1791792513.GET./api/v1/payment/order/ORD-DOC-001.
X-Signature :
159ab4b58d93e2ac834c33312f43550fbe5942d56324a1f005141a216232b883
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "transaction": {
    "id": "d3m00000000000000000000000000001",
    "ref": "TXN100011",
    "merchantId": "6",
    "orderId": "ORD-DOC-001",
    "amount": 100,
    "fee": 1.5,
    "netAmount": 98.5,
    "status": "PENDING",
    "source": "API",
    "paymentMethod": "BANK_TRANSFER",
    "description": "สินค้า A",
    "qrCode": null,
    "qrCodeUrl": null,
    "qrExpiry": "2026-10-12T08:23:26.830Z",
    "checkoutUrl": "https://optionpay.co/c?
t=d3m00000000000000000000000000001",
    "createdAt": "2026-10-12T08:08:26.830Z",
    "paidAt": null,
    "copy": {
      "amount": "100.00"
    }
  }
}
```


404 ORDER_NOT_FOUND ถ้าไม่พบ

GET /payment/checkout/:transactionId

Checkout Page Data (ไม่ต้องส่ง API Key)

ไม่ต้องใช้สิทธิ์ AUTH: ไม่ต้องมี key — transaction id เป็นความลับของลูกค้าคนนั้น

ตัวอย่าง — CURL

```
# ไม่ต้องมี key / ลายเซ็น — transaction id เป็นความลับของลูกค้าคนนั้น
curl
"https://optionpay.co/api/v1/payment/checkout/d3m00000000000000000000000000
001"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler · endpoint สาธารณะ ไม่
ต้องมี key / ลายเซ็น
export async function GET() {
  const res = await
```

```
fetch("https://optionpay.co/api/v1/payment/checkout/d3m00000000000000000000000001");
  return Response.json(await res.json(), { status: res.status });
}
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "transaction": {
    "id": "d3m00000000000000000000000000001",
    "orderId": "ORD-DOC-001",
    "amount": 100,
    "status": "PENDING",
    "description": "สินค้า A",
    "qrExpiry": "2026-10-12T08:23:26.830Z",
    "createdAt": "2026-10-12T08:08:26.830Z",
    "paidAt": null,
    "copy": {
      "amount": "100.00"
    },
    "paymentMethod": "BANK_TRANSFER"
  },
  "merchant": {
    "name": "your-shop"
  },
  "account": {
    "bankCode": "SCB",
    "bankName": "ธนาคารไทยพาณิชย์",
    "accountName": "บริษัท ตัวอย่าง จำกัด",
    "accountNumber": "1234567890",
    "promptPayId": null
  },
  "qrPayload": null,
  "customerName": "สมชาย ใจดี",
  "hasSlip": false,
  "slipLocked": false,
  "slipProblems": null
}
```


หน้าชำระเงินสำเร็จรูปสำหรับลูกค้า: `https://optionpay.co/c?t={transactionId}` — แสดงยอด บัญชีรับเงิน นับถอยหลัง แนบสลิป แนวทางการแนบสลิป ปุ่มดาวน์โหลดตัวอย่างสลิป (JPG) และอัปเดตสถานะอัตโนมัติ

POST /payment/checkout/:transactionId/slip

ลูกค้าแนบสลิปจากหน้าชำระเงิน (ไม่ต้องส่ง API Key)

ไม่ต้องใช้สิทธิ์ AUTH: ไม่ต้องมี key — transaction id เป็นความลับของลูกค้าคนนั้น

คำอธิบายฟิลด์

```
{ "image": "<base64>", "mime": "image/jpeg" }   // png | jpeg | webp, ไม่เกิน
8 MB
```

ตัวอย่าง — CURL

```
# ไม่ต้องมี key / ลายเซ็น — transaction id เป็นความลับของลูกค้าคนนั้น
BODY="{\"mime\":\"image/png\",\"image\":\"$(base64 < slip2.png | tr -d
'\n')\"}"   # slip2.png = รูปสลิป
curl -X POST
"https://optionpay.co/api/v1/payment/checkout/d3m00000000000000000000000000
001/slip" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler · endpoint สาธารณะ ไม่
ต้องมี key / ลายเซ็น
import { readFile } from "node:fs/promises";

export async function POST() {
  const res = await
fetch("https://optionpay.co/api/v1/payment/checkout/d3m00000000000000000000
000000001/slip", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
    "mime": "image/png",
    "image": (await readFile("slip2.png")).toString("base64")
  }),
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "verified": false,
  "result": "PENDING_REVIEW",
  "message": "ไม่ได้เปิดการตรวจสลิป — รอผู้ดูแลตรวจ",
  "mismatches": [],
  "status": "PENDING"
}
```

ตรวจตามกฎเดียวกับ /payment/slip/verify · **แนบได้ครั้งเดียวต่อรายการ** — มีสลิปที่รับแล้ว/รอตรวจอยู่ → `409 SLIP_ALREADY_UPLOADED` (แนบใหม่ได้เฉพาะเมื่อสลิปก่อนหน้าไม่ผ่าน) · สลิปตัวอย่าง → `400 SAMPLE_SLIP`

GET /payment/qr/:token.png

รูป QR ของออเดอร์ (PNG, ไม่ต้องส่ง API Key)

ไม่ต้องใช้สิทธิ์ AUTH: ไม่ต้องมี key

ตัวอย่าง — CURL

```
# ไม่ต้องมี key / ลายเซ็น — transaction id เป็นความลับของลูกค้าคนนั้น
curl
"https://optionpay.co/api/v1/payment/qr/d3m00000000000000000000000000001.pn
g" \
  -o qr.png
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler · endpoint สาธารณะ ไม่
ต้องมี key / ลายเซ็น
export async function GET() {
  const res = await
fetch("https://optionpay.co/api/v1/payment/qr/d3m00000000000000000000000000
001.png");
  return new Response(await res.arrayBuffer(), { status: res.status,
headers: { "Content-Type": res.headers.get("content-type") ?? "image/png" }
});
}
```

ผลลัพธ์ — HTTP 410

```
(text/plain;charset=UTF-8, 74 bytes)
```

ตอบ `410` เมื่อออเดอร์ไม่ PENDING แล้ว หรือบัญชีรับเงินไม่มี PromptPay

POST /collect

สร้าง Payment (ชื่อเดิม) — เหมือน /payment/create

สิทธิ์: **create_payment** · สร้างรายการชำระเงิน และแนบ/ตรวจสลิป

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

คำอธิบายฟิลด์

```
{ "amount": 150, "orderId": "ORD-004" }   // ไม่มีฟิลด์ key ใน body แล้ว
```

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792518"   # ปกติใช้ TS=$(date +%s)
BODY='{"amount":150,"orderId":"ORD-DOC-004"}'
SIG=$(printf '%s' "$TS.POST./api/v1/collect.$BODY" | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = c270b8f6b94e105c2ef7a2913ee49982799b56967184524dc2f81803133777d6

curl -X POST "https://optionpay.co/api/v1/collect" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
  const path = "/api/v1/collect";
  const body = JSON.stringify({
    "amount": 150,
    "orderId": "ORD-DOC-004"
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792518"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                                  // ค่าตัวอย่าง: "c270b8f6b94e105c2ef7a2913ee49982799b56967184524dc2f81803133777d6"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
```

```
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792518
สตริงที่เซ็น : 1791792518.POST./api/v1/collect.{"amount":150,"orderId":"ORD-
DOC-004"}
X-Signature :
c270b8f6b94e105c2ef7a2913ee49982799b56967184524dc2f81803133777d6
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "orderIdGenerated": false,
  "transaction": {
    "id": "19f96c6e2d7b1c33bf24ed3c38908eb0",
    "ref": "TXN100014",
    "merchantId": "6",
    "orderId": "ORD-DOC-004",
    "amount": 150,
    "fee": 2.25,
    "netAmount": 147.75,
    "status": "PENDING",
    "source": "COLLECT",
    "paymentMethod": "BANK_TRANSFER",
    "description": null,
    "qrCode": null,
    "qrCodeUrl": null,
    "qrExpiry": "2026-10-12T08:23:39.021Z",
    "checkoutUrl": "https://optionpay.co/c?
t=19f96c6e2d7b1c33bf24ed3c38908eb0",
    "createdAt": "2026-10-12T08:08:39.021Z",
    "paidAt": null,
    "copy": {
      "amount": "150.00"
    }
  },
  "selectedAccount": {
    "id": "8",
    "type": "BANK",
    "accountNumber": "1234567890",
    "accountName": "บริษัท ตัวอย่าง จำกัด",
    "bankCode": "SCB",
    "bankName": "ธนาคารไทยพาณิชย์"
```

```
  }
}
```

> ใช้ header และลายเซ็นเหมือนทุก endpoint (`X-API-Key` + `X-Timestamp` + `X-Signature`) · ส่ง `key` ใน body ไม่ได้แล้ว · ระบบใหม่ใช้ `/payment/create` แทนได้เลย

Transactions (ประวัติ)

GET **/transactions**

รายการ Transactions ทั้งหมด

สิทธิ์: **view_transactions** · ดูรายการชำระเงิน / payout / คำขอถอน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792520"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/transactions?limit=2&offset=0&status=PENDING." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = c6bc15b559c0a79f148812be33cb28410768b8d0562afd0cfc5a215746f16717

curl "https://optionpay.co/api/v1/transactions?limit=2&offset=0&status=PENDING" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/transactions?limit=2&offset=0&status=PENDING";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792520"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง: "c6bc15b559c0a79f148812be33cb28410768b8d0562afd0cfc5a215746f16717"

  const res = await fetch(`https://optionpay.co${path}`, {
```

```
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792520
สตริงที่เซ็น : 1791792520.GET./api/v1/transactions?
limit=2&offset=0&status=PENDING.
X-Signature :
c6bc15b559c0a79f148812be33cb28410768b8d0562afd0cfc5a215746f16717
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "data": [
    {
      "id": "19f96c6e2d7b1c33bf24ed3c38908eb0",
      "ref": "TXN100014",
      "merchantId": "6",
      "orderId": "ORD-DOC-004",
      "amount": 150,
      "fee": 2.25,
      "netAmount": 147.75,
      "status": "PENDING",
      "source": "COLLECT",
      "paymentMethod": "BANK_TRANSFER",
      "description": null,
      "qrCode": null,
      "qrCodeUrl": null,
      "qrExpiry": "2026-10-12T08:23:39.021Z",
      "checkoutUrl": "https://optionpay.co/c?
t=19f96c6e2d7b1c33bf24ed3c38908eb0",
      "createdAt": "2026-10-12T08:08:39.021Z",
      "paidAt": null,
      "copy": {
        "amount": "150.00"
      }
    },
    {
      "id": "cd6584e2787b93f451d9c47b4513c1df",
      "ref": "TXN100013",
      "merchantId": "6",
      "orderId": "ORD-DOC-003",
      "amount": 300,
```

```
      "fee": 4.5,
      "netAmount": 295.5,
      "status": "PENDING",
      "source": "SLIP",
      "paymentMethod": "BANK_TRANSFER",
      "description": "สินค้า C",
      "qrCode": null,
      "qrCodeUrl": null,
      "qrExpiry": "2026-10-12T08:23:30.971Z",
      "checkoutUrl": "https://optionpay.co/c?
t=cd6584e2787b93f451d9c47b4513c1df",
      "createdAt": "2026-10-12T08:08:30.971Z",
      "paidAt": null,
      "copy": {
        "amount": "300.00"
      }
    }
  ],
  "pagination": {
    "total": 4,
    "limit": 2,
    "offset": 0
  }
}
```


Query: `limit` (1–200, ค่าเริ่มต้น 50) · `offset` · `status` (คั่นด้วย , ได้) · `from` / `to` (ISO 8601 พร้อม timezone)

GET /transactions/:id

ดู Transaction ตาม transaction id (หรือ ref TXN...)

สิทธิ์: **view_transactions** · ดูรายการชำระเงิน / payout / คำขอถอน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น
ห้ามส่ง
TS="1791792521"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s'
"$TS.GET./api/v1/transactions/d3m00000000000000000000000000001." | openssl
dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 84863d5b4a966d54c65806a1aebd12a8489067ebbb6a7a20a9500a46cee9a681

curl
"https://optionpay.co/api/v1/transactions/d3m00000000000000000000000000001"
\
```

```
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/transactions/d3m00000000000000000000000000001";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792521"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง: "84863d5b4a966d54c65806a1aebd12a8489067ebbb6a7a20a9500a46cee9a681"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792521
สตริงที่เซ็น :
1791792521.GET./api/v1/transactions/d3m00000000000000000000000000001.
X-Signature :
84863d5b4a966d54c65806a1aebd12a8489067ebbb6a7a20a9500a46cee9a681
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
```

```
  "transaction": {
    "id": "d3m00000000000000000000000000001",
    "ref": "TXN100011",
    "merchantId": "6",
    "orderId": "ORD-DOC-001",
    "amount": 100,
    "fee": 1.5,
    "netAmount": 98.5,
    "status": "PENDING",
    "source": "API",
    "paymentMethod": "BANK_TRANSFER",
    "description": "สินค้า A",
    "qrCode": null,
    "qrCodeUrl": null,
    "qrExpiry": "2026-10-12T08:23:26.830Z",
    "checkoutUrl": "https://optionpay.co/c?
t=d3m00000000000000000000000000001",
    "createdAt": "2026-10-12T08:08:26.830Z",
    "paidAt": null,
    "copy": {
      "amount": "100.00"
    }
  }
}
```


PATCH /transactions/:id/cancel

ยกเลิก Transaction

สิทธิ์: **cancel_payment** · ยกเลิกรายการชำระเงิน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น
ห้ามส่ง
TS="1791792522"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s'
"$TS.PATCH./api/v1/transactions/19f96c6e2d7b1c33bf24ed3c38908eb0/cancel." |
openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 0984af18a1595b0e0994e32d483a3f9228a311545fd93efcacbf84624cb79fe3

curl -X PATCH
"https://optionpay.co/api/v1/transactions/19f96c6e2d7b1c33bf24ed3c38908eb0/
cancel" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "PATCH";
  const path = "/api/v1/transactions/19f96c6e2d7b1c33bf24ed3c38908eb0/cancel";
  const body = ""; // PATCH ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792522"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง: "0984af18a1595b0e0994e32d483a3f9228a311545fd93efcacbf84624cb79fe3"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792522
สตริงที่เซ็น :
1791792522.PATCH./api/v1/transactions/19f96c6e2d7b1c33bf24ed3c38908eb0/cancel.
X-Signature :
0984af18a1595b0e0994e32d483a3f9228a311545fd93efcacbf84624cb79fe3
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "transaction": {
    "id": "19f96c6e2d7b1c33bf24ed3c38908eb0",
    "ref": "TXN100014",
```

```
    "merchantId": "6",
    "orderId": "ORD-DOC-004",
    "amount": 150,
    "fee": 2.25,
    "netAmount": 147.75,
    "status": "CANCELLED",
    "source": "COLLECT",
    "paymentMethod": "BANK_TRANSFER",
    "description": null,
    "qrCode": null,
    "qrCodeUrl": null,
    "qrExpiry": "2026-10-12T08:23:39.021Z",
    "checkoutUrl": "https://optionpay.co/c?
t=19f96c6e2d7b1c33bf24ed3c38908eb0",
    "createdAt": "2026-10-12T08:08:39.021Z",
    "paidAt": null,
    "copy": {
      "amount": "150.00"
    }
  }
}
```


> ยกเลิกได้เฉพาะ PENDING / EXPIRED — อื่น ๆ ได้ `409 ORDER_NOT_PENDING` · ยิง webhook `payment.rejected`

Payout (โอนเงินออก)

สั่งโอนเงินออกจากยอดของร้าน — **อนุมัติอัตโนมัติทันทีที่สร้าง**: ตอบ `status: COMPLETED` (อนุมัติแล้ว) พร้อม `completedAt` และ**หักยอด amount + fee จากร้านทันที** · ทีม Option Pay โอนเงินให้ผู้รับต่อจากนั้น (ติดตามที่หน้ารายการเงินออกของผู้ดูแล) · สถานะไม่เปลี่ยนอีก — ไม่มีการคืนยอดอัตโนมัติ · `/payout/create` และ `/payout/bulk` ทำงานแบบเดียวกัน · ร้านดูรายการได้ที่ Merchant Portal → **รายการโอนออก** และ `GET /payout/:id` / `GET /payout/ref/:merchantRef` · ไม่มี webhook สำหรับ payout

POST /payout/create

สร้าง Payout — รองรับ Idempotency-Key header

สิทธิ์: **create_payout** · สั่งจ่ายเงินออก (payout)

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature` + `Idempotency-Key` (optional)

คำอธิบายฟิลด์

```
{
  "amount": 500.00,             // required
  "bankCode": "SCB",            // required — รหัสธนาคาร (31 ธนาคาร): SCB
BAY KKP KTB UOB HSBC GHB TISCO BBL KBANK CIMB GSB BAAC LHBANK MHCB TTB CITI
SCBT ISBT ACL TCRB ANZ SMTB SMEB ABN TRUEMONEY ICBC SMBC BOC DB IOB
  "accountNumber": "1234567890",// required, 6–20 หลัก — ใส่ขีด/เว้นวรรคได้ (เช่น
"123-4-56789-0") ระบบตัดออกให้
  "accountName": "นาย ก",        // required
  "merchantRef": "PO-001",      // optional, unique ต่อ merchant
  "description": "ค่าสินค้า"      // optional
}
```

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น
ห้ามส่ง
TS="1791792524"   # ปกติใช้ TS=$(date +%s)
BODY='{"amount":10,"bankCode":"SCB","accountNumber":"1234567890","accountNa
me":"สมชาย ใจดี","merchantRef":"PO-DOC-001","description":"ค่าสินค้า"}'
SIG=$(printf '%s' "$TS.POST./api/v1/payout/create.$BODY" | openssl dgst -
sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = e0935ac46a504a33816003dd0ab39113246a481416b39b7ffc23909761a42e76

curl -X POST "https://optionpay.co/api/v1/payout/create" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "Idempotency-Key: po-doc-001" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
  const path = "/api/v1/payout/create";
  const body = JSON.stringify({
    "amount": 10,
    "bankCode": "SCB",
    "accountNumber": "1234567890",
    "accountName": "สมชาย ใจดี",
    "merchantRef": "PO-DOC-001",
    "description": "ค่าสินค้า"
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792524"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                                  // ค่าตัวอย่าง: "e0935ac46a504a33816003dd0ab39113246a481416b39b7ffc23909761a42e76"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Idempotency-Key": "po-doc-001",
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792524
สตริงที่เซ็น : 1791792524.POST./api/v1/payout/create.
{"amount":10,"bankCode":"SCB","accountNumber":"1234567890","accountName":"สมชาย ใจดี","merchantRef":"PO-DOC-001","description":"ค่าสินค้า"}
```

```
X-Signature :
e0935ac46a504a33816003dd0ab39113246a481416b39b7ffc23909761a42e76
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "payout": {
    "id": "OUT10152",
    "merchantRef": "PO-DOC-001",
    "amount": 10,
    "status": "COMPLETED",
    "bankCode": "SCB",
    "accountNumber": "1234567890",
    "accountName": "สมชาย ใจดี",
    "description": "ค่าสินค้า",
    "createdAt": "2026-10-12T08:08:44.397Z",
    "completedAt": "2026-10-12T08:08:44.397Z",
    "failureReason": null
  }
}
```


ส่ง Idempotency-Key เดิมซ้ำ = ได้ payout เดิมกลับมา (`idempotent: true`) ไม่สร้างซ้ำ · **Errors**: ยอดใช้ได้ไม่พอ → `400 INSUFFICIENT_BALANCE` · ทำให้เหลือต่ำกว่ายอดคงเหลือขั้นต่ำ → `400 below_min_balance` · ต่ำกว่ายอดถอนขั้นต่ำ → `400 below_min_amount` · merchantRef ซ้ำ → `409 DUPLICATE_MERCHANT_REF`

GET /payout/:id

ดู Payout ตาม id (OUT...)

สิทธิ์: **view_transactions** · ดูรายการชำระเงิน / payout / คำขอถอน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792525"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/payout/OUT10152." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = ce0397c7c73e151f8ea240516addcd821988cebfa871f01e233e3d652f354811

curl "https://optionpay.co/api/v1/payout/OUT10152" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/payout/OUT10152";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792525"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง: "ce0397c7c73e151f8ea240516addcd821988cebfa871f01e233e3d652f354811"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792525
สตริงที่เซ็น : 1791792525.GET./api/v1/payout/OUT10152.
X-Signature :
ce0397c7c73e151f8ea240516addcd821988cebfa871f01e233e3d652f354811
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "payout": {
    "id": "OUT10152",
    "merchantRef": "PO-DOC-001",
    "amount": 10,
    "status": "COMPLETED",
    "bankCode": "SCB",
```

```
    "accountNumber": "1234567890",
    "accountName": "สมชาย ใจดี",
    "description": "ค่าสินค้า",
    "createdAt": "2026-10-12T08:08:44.397Z",
    "completedAt": "2026-10-12T08:08:44.397Z",
    "failureReason": null
  }
}
```

GET /payout/ref/:merchantRef

ดู Payout ตาม merchantRef

สิทธิ์: **view_transactions** · ดูรายการชำระเงิน / payout / คำขอถอน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792526"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/payout/ref/PO-DOC-001." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 26501e494ea595e71a09ff4cd7c889c111c90f247edc76b3775f5e5c9588a885

curl "https://optionpay.co/api/v1/payout/ref/PO-DOC-001" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/payout/ref/PO-DOC-001";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792526"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
```

```
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง:
"26501e494ea595e71a09ff4cd7c889c111c90f247edc76b3775f5e5c9588a885"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792526
สตริงที่เซ็น : 1791792526.GET./api/v1/payout/ref/PO-DOC-001.
X-Signature :
26501e494ea595e71a09ff4cd7c889c111c90f247edc76b3775f5e5c9588a885
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "payout": {
    "id": "OUT10152",
    "merchantRef": "PO-DOC-001",
    "amount": 10,
    "status": "COMPLETED",
    "bankCode": "SCB",
    "accountNumber": "1234567890",
    "accountName": "สมชาย ใจดี",
    "description": "ค่าสินค้า",
    "createdAt": "2026-10-12T08:08:44.397Z",
    "completedAt": "2026-10-12T08:08:44.397Z",
    "failureReason": null
  }
}
```

GET /payouts

รายการ Payouts

สิทธิ์: **view_transactions** · ดูรายการชำระเงิน / payout / คำขอถอน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792528"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/payouts?limit=5&offset=0." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = ecf85cbcd2bbc4ad43528595850ac6fa0cfa1d057318348aeb371e446847a8db

curl "https://optionpay.co/api/v1/payouts?limit=5&offset=0" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/payouts?limit=5&offset=0";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792528"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง: "ecf85cbcd2bbc4ad43528595850ac6fa0cfa1d057318348aeb371e446847a8db"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792528
สตริงที่เซ็น : 1791792528.GET./api/v1/payouts?limit=5&offset=0.
X-Signature :
ecf85cbcd2bbc4ad43528595850ac6fa0cfa1d057318348aeb371e446847a8db
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "data": [
    {
      "id": "OUT10152",
      "merchantRef": "PO-DOC-001",
      "amount": 10,
      "status": "COMPLETED",
      "bankCode": "SCB",
      "accountNumber": "1234567890",
      "accountName": "สมชาย ใจดี",
      "description": "ค่าสินค้า",
      "createdAt": "2026-10-12T08:08:44.397Z",
      "completedAt": "2026-10-12T08:08:44.397Z",
      "failureReason": null
    }
  ],
  "pagination": {
    "total": 1,
    "limit": 5,
    "offset": 0
  }
}
```


> Query: `limit` (1–200, ค่าเริ่มต้น 50) · `offset` · `status` (คั่นด้วย , ได้) · `from` / `to` (ISO 8601 พร้อม timezone)

POST /payout/bulk

Bulk Payout — สูงสุด 100 รายการ/ครั้ง

สิทธิ์: **create_payout** · สั่งจ่ายเงินออก (payout)

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

คำอธิบายฟิลด์

```
{ "payouts": [ { ...เหมือน /payout/create... }, ... ] }
```

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792529"   # ปกติใช้ TS=$(date +%s)
BODY='{"payouts":
[{"amount":11,"bankCode":"SCB","accountNumber":"1234567890","accountName":"สมชาย ใจดี","merchantRef":"PO-DOC-101"},
{"amount":12,"bankCode":"KBANK","accountNumber":"9876543210","accountName":"มาลี สวยงาม","merchantRef":"PO-DOC-102"}]}'
SIG=$(printf '%s' "$TS.POST./api/v1/payout/bulk.$BODY" | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 73ba8e1ee620a96a32b2dae687c9f4451aa269b2dc89c07d27f39995d9552de9

curl -X POST "https://optionpay.co/api/v1/payout/bulk" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
  const path = "/api/v1/payout/bulk";
  const body = JSON.stringify({
    "payouts": [
      {
        "amount": 11,
        "bankCode": "SCB",
        "accountNumber": "1234567890",
        "accountName": "สมชาย ใจดี",
        "merchantRef": "PO-DOC-101"
      },
      {
        "amount": 12,
        "bankCode": "KBANK",
        "accountNumber": "9876543210",
        "accountName": "มาลี สวยงาม",
        "merchantRef": "PO-DOC-102"
      }
    ]
```

```
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง:
"1791792529"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                                  // ค่าตัวอย่าง:
"73ba8e1ee620a96a32b2dae687c9f4451aa269b2dc89c07d27f39995d9552de9"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792529
สตริงที่เซ็น : 1791792529.POST./api/v1/payout/bulk.{"payouts":
[{"amount":11,"bankCode":"SCB","accountNumber":"1234567890","accountName":"
สมชาย ใจดี","merchantRef":"PO-DOC-101"},
{"amount":12,"bankCode":"KBANK","accountNumber":"9876543210","accountName":
"มาลี สวยงาม","merchantRef":"PO-DOC-102"}]}
X-Signature :
73ba8e1ee620a96a32b2dae687c9f4451aa269b2dc89c07d27f39995d9552de9
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "created": 2,
  "failed": 0,
  "results": [
    {
      "index": 0,
      "success": true,
      "payout": {
        "id": "OUT10153",
        "merchantRef": "PO-DOC-101",
        "amount": 11,
        "status": "COMPLETED",
        "bankCode": "SCB",
        "accountNumber": "1234567890",
        "accountName": "สมชาย ใจดี",
```

```
        "description": "Payout ผ่าน API (PO-DOC-101)",
        "createdAt": "2026-10-12T08:08:49.762Z",
        "completedAt": "2026-10-12T08:08:49.762Z",
        "failureReason": null
      }
    },
    {
      "index": 1,
      "success": true,
      "payout": {
        "id": "OUT10154",
        "merchantRef": "PO-DOC-102",
        "amount": 12,
        "status": "COMPLETED",
        "bankCode": "KBANK",
        "accountNumber": "9876543210",
        "accountName": "มาลี สวยงาม",
        "description": "Payout ผ่าน API (PO-DOC-102)",
        "createdAt": "2026-10-12T08:08:49.778Z",
        "completedAt": "2026-10-12T08:08:49.778Z",
        "failureReason": null
      }
    }
  ]
}
```


> แต่ละรายการทำแยกกัน — รายการที่ผ่านไม่ถูกยกเลิกเพราะรายการอื่นล้ม

Withdrawal Request (คำขอถอน)

คำขอถอนเงินเข้าบัญชีของร้าน — ผู้ดูแลอนุมัติที่หน้า **คำขอถอนเงิน** · สถานะ: `PENDING | APPROVED | REJECTED`

POST /withdrawal/request/create

สร้างคำขอถอนเงิน

สิทธิ์: **request_withdrawal** · ขอถอนเงิน AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

คำอธิบายฟิลด์

```
{ "amount": 1000, "bankCode": "KBANK", "accountNumber": "9876543210",
"accountName": "นาย ก",
  // bankCode: รหัสธนาคารชุดเดียวกับ /payout/create (31 ธนาคาร) · accountNumber
6–20 หลัก ใส่ขีด/เว้นวรรคได้
  "merchantRef": "W-001",   // optional, unique ต่อ merchant
  "note": "ถอนประจำวัน" }
```

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น
ห้ามส่ง
TS="1791792530"   # ปกติใช้ TS=$(date +%s)
BODY='{"amount":100,"bankCode":"KBANK","accountNumber":"9876543210","accoun
tName":"สมชาย ใจดี","merchantRef":"W-DOC-001","note":"ถอนประจำวัน"}'
SIG=$(printf '%s' "$TS.POST./api/v1/withdrawal/request/create.$BODY" |
openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 5fd9c50b066ecfdda60adc08c849cbe3eb1623a84a5cfd3be0af79e8a90596dc

curl -X POST "https://optionpay.co/api/v1/withdrawal/request/create" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น
ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง:
"ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง:
"sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
```

```
  const path = "/api/v1/withdrawal/request/create";
  const body = JSON.stringify({
    "amount": 100,
    "bankCode": "KBANK",
    "accountNumber": "9876543210",
    "accountName": "สมชาย ใจดี",
    "merchantRef": "W-DOC-001",
    "note": "ถอนประจำวัน"
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง:
"1791792530"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง:
"5fd9c50b066ecfdda60adc08c849cbe3eb1623a84a5cfd3be0af79e8a90596dc"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792530
สตริงที่เซ็น : 1791792530.POST./api/v1/withdrawal/request/create.
{"amount":100,"bankCode":"KBANK","accountNumber":"9876543210","accountName"
:"สมชาย ใจดี","merchantRef":"W-DOC-001","note":"ถอนประจำวัน"}
X-Signature :
5fd9c50b066ecfdda60adc08c849cbe3eb1623a84a5cfd3be0af79e8a90596dc
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "request": {
    "requestRef": "WDR10001",
    "merchantRef": "W-DOC-001",
    "amount": 100,
    "status": "PENDING",
    "bankCode": "KBANK",
    "accountNumber": "9876543210",
    "accountName": "สมชาย ใจดี",
```

```
    "note": "ถอนประจำวัน",
    "rejectReason": null,
    "createdAt": "2026-10-12T08:08:51.206Z",
    "decidedAt": null
  }
}
```

> ตรวจยอดใช้ได้ / ยอดคงเหลือขั้นต่ำ / ยอดถอนขั้นต่ำ เหมือน payout

GET /withdrawal/requests

รายการคำขอถอนเงิน

สิทธิ์: **view_transactions** · ดูรายการชำระเงิน / payout / คำขอถอน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792532"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/withdrawal/requests?limit=5&offset=0." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = a798194689caa738b03ca632971257bd91ec089415540099ab31ebe09358ea40

curl "https://optionpay.co/api/v1/withdrawal/requests?limit=5&offset=0" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/withdrawal/requests?limit=5&offset=0";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792532"
```

```
  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง:
"a798194689caa738b03ca632971257bd91ec089415540099ab31ebe09358ea40"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792532
สตริงที่เซ็น : 1791792532.GET./api/v1/withdrawal/requests?limit=5&offset=0.
X-Signature :
a798194689caa738b03ca632971257bd91ec089415540099ab31ebe09358ea40
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "data": [
    {
      "requestRef": "WDR10001",
      "merchantRef": "W-DOC-001",
      "amount": 100,
      "status": "PENDING",
      "bankCode": "KBANK",
      "accountNumber": "9876543210",
      "accountName": "สมชาย ใจดี",
      "note": "ถอนประจำวัน",
      "rejectReason": null,
      "createdAt": "2026-10-12T08:08:51.206Z",
      "decidedAt": null
    }
  ],
  "pagination": {
    "total": 1,
    "limit": 5,
    "offset": 0
  }
}
```

Query: `limit` (1–200, ค่าเริ่มต้น 50) · `offset` · `status` (คั่นด้วย , ได้) · `from` / `to` (ISO 8601 พร้อม timezone)

GET /withdrawal/request/:ref

ดูคำขอตาม requestRef หรือ merchantRef — shape เดิม

สิทธิ์: **view_transactions** · ดูรายการชำระเงิน / payout / คำขอถอน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792533"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/withdrawal/request/WDR10001." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = cb9cb646afaaf9aa8243f2f0ca433c7216169008c8eed4e0107c1d26ad0da3b9

curl "https://optionpay.co/api/v1/withdrawal/request/WDR10001" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/withdrawal/request/WDR10001";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792533"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง: "cb9cb646afaaf9aa8243f2f0ca433c7216169008c8eed4e0107c1d26ad0da3b9"
```

```
  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792533
สตริงที่เซ็น : 1791792533.GET./api/v1/withdrawal/request/WDR10001.
X-Signature :
cb9cb646afaaf9aa8243f2f0ca433c7216169008c8eed4e0107c1d26ad0da3b9
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "request": {
    "requestRef": "WDR10001",
    "merchantRef": "W-DOC-001",
    "amount": 100,
    "status": "PENDING",
    "bankCode": "KBANK",
    "accountNumber": "9876543210",
    "accountName": "สมชาย ใจดี",
    "note": "ถอนประจำวัน",
    "rejectReason": null,
    "createdAt": "2026-10-12T08:08:51.206Z",
    "decidedAt": null
  }
}
```

Payment Accounts (บัญชีรับเงิน)

บัญชีรับเงินของร้าน — 1 บัญชีใช้ร่วมได้หลายร้าน (ผู้ดูแลระบบเป็นผู้ผูก): `shared: true` = บัญชีนี้ร้านอื่นใช้ด้วย · การใช้บัญชีร่วม**ไม่เปลี่ยนยอด** — `amount` เท่ากับที่ส่งมา (บวกสตางค์เฉพาะเมื่อผู้ดูแลเปิด "ยอดไม่ซ้ำ" ในตั้งค่าระบบ — แสดง `copy.amount` จาก response ให้ลูกค้าเสมอ) · ร้านย่อยที่ไม่มีบัญชีของตัวเองใช้บัญชีของร้านแม่ · ถ้าร้านไม่มีบัญชีของตัวเอง ออเดอร์จะใช้บัญชีกลางของระบบ (เมื่อเปิดโหมดบัญชีรับเงินกลาง)

GET /payment/accounts

รายการบัญชีรับเงิน

สิทธิ์: **manage_accounts** · จัดการบัญชีรับเงิน AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792534"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/payment/accounts." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = b9fed33475bdea88052262d762a75fb8e56ec1d3fc7c9fd2b1e6666c9c4a5267

curl "https://optionpay.co/api/v1/payment/accounts" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/payment/accounts";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792534"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                                  // ค่าตัวอย่าง: "b9fed33475bdea88052262d762a75fb8e56ec1d3fc7c9fd2b1e6666c9c4a5267"
```

```
  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792534
สตริงที่เซ็น : 1791792534.GET./api/v1/payment/accounts.
X-Signature :
b9fed33475bdea88052262d762a75fb8e56ec1d3fc7c9fd2b1e6666c9c4a5267
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "data": [
    {
      "id": "8",
      "type": "BANK",
      "accountNumber": "1234567890",
      "accountName": "บริษัท ตัวอย่าง จำกัด",
      "bankCode": "SCB",
      "bankName": "ธนาคารไทยพาณิชย์",
      "status": "ACTIVE"
    }
  ],
  "centralReceiving": true,
  "note": null
}
```

POST /payment/accounts

เพิ่มบัญชีรับเงิน — ถ้าบัญชีนี้ผูกกับร้านนี้อยู่แล้ว จะคืนบัญชีเดิม · ถ้าเป็นบัญชีของร้านอื่น / บัญชีกลาง ได้ 409 ACCOUNT_EXISTS (ให้ผู้ดูแลระบบผูกให้ — ใช้ร่วมกันได้)

สิทธิ์: **manage_accounts** · จัดการบัญชีรับเงิน AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

คำอธิบายฟิลด์

```
{
  "type": "BANK",                 // BANK | PROMPTPAY
  "bankCode": "SCB",              // required สำหรับ BANK
```

```
  "accountNumber": "1234567890",  // required สำหรับ BANK
  "accountName": "บจก. เอบีซี",
  "promptPayId": "0812345678"     // required สำหรับ PROMPTPAY (มือถือ 10 หลัก
/ เลขประจำตัว 13 หลัก)
}
```

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น
ห้ามส่ง
TS="1791792536"   # ปกติใช้ TS=$(date +%s)
BODY='{"type":"PROMPTPAY","promptPayId":"0812345678","accountName":"บจก. เอ
บีซี"}'
SIG=$(printf '%s' "$TS.POST./api/v1/payment/accounts.$BODY" | openssl dgst
-sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 94f903882fa356cc14cbb1637998edd6965d6e3bc8785c39e8a389c0b07a97be

curl -X POST "https://optionpay.co/api/v1/payment/accounts" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น
ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง:
"ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง:
"sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
  const path = "/api/v1/payment/accounts";
  const body = JSON.stringify({
    "type": "PROMPTPAY",
    "promptPayId": "0812345678",
    "accountName": "บจก. เอบีซี"
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง:
"1791792536"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง:
"94f903882fa356cc14cbb1637998edd6965d6e3bc8785c39e8a389c0b07a97be"
```

```
  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792536
สตริงที่เซ็น : 1791792536.POST./api/v1/payment/accounts.
{"type":"PROMPTPAY","promptPayId":"0812345678","accountName":"บจก. เอบีซี"}
X-Signature :
94f903882fa356cc14cbb1637998edd6965d6e3bc8785c39e8a389c0b07a97be
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "account": {
    "id": "10",
    "type": "PROMPTPAY",
    "accountNumber": "0812345678",
    "accountName": "บจก. เอบีซี",
    "bankCode": "KBANK",
    "bankName": "ธนาคารกสิกรไทย",
    "status": "ACTIVE"
  }
}
```

DELETE /payment/accounts/:id

ลบบัญชีรับเงิน (soft delete → inactive) · บัญชีที่ใช้ร่วมกับร้านอื่น: เลิกผูกเฉพาะร้านนี้ (บัญชียังใช้งานกับร้านอื่น) — ทำไม่ได้ถ้าร้านนี้ยังมีออเดอร์รอชำระบนบัญชีนั้น (409 ACCOUNT_HAS_PENDING_ORDERS) — ยกเว้นบัญชีที่ปิดใช้งานแล้ว

สิทธิ์: **manage_accounts** · จัดการบัญชีรับเงิน AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น
ห้ามส่ง
```

```
TS="1791792537"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.DELETE./api/v1/payment/accounts/10." | openssl dgst
-sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 26af42dcc4e2c398bdcabfff3e9a111f33bb21880a0e5f2635e117ca50ef8b34

curl -X DELETE "https://optionpay.co/api/v1/payment/accounts/10" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น
ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง:
"ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง:
"sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "DELETE";
  const path = "/api/v1/payment/accounts/10";
  const body = ""; // DELETE ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง:
"1791792537"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                              // ค่าตัวอย่าง:
"26af42dcc4e2c398bdcabfff3e9a111f33bb21880a0e5f2635e117ca50ef8b34"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792537
สตริงที่เซ็น : 1791792537.DELETE./api/v1/payment/accounts/10.
X-Signature :
26af42dcc4e2c398bdcabfff3e9a111f33bb21880a0e5f2635e117ca50ef8b34
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true
}
```

Webhooks

ระบบยิง `POST` JSON ไปที่ URL ของร้านเมื่อสถานะเปลี่ยน — header 4 ตัวเท่านั้น (ชื่อตัวพิมพ์เล็ก): `content-type: application/json` , `user-agent: OptionPay-Webhook/1.0` , `x-webhook-event: <event>` , `x-signature` (คำสั่ง curl ของแต่ละ event ดูหัวข้อ “Callback ที่ระบบส่งถึงร้าน” ในคู่มือ API ของ Merchant Portal) พร้อม header `X-Signature: t=<unix>,v1=<hex>` — เซ็นด้วย **secret key ของร้าน** (`sk_live_…`) 2 ขั้น: `key = HMAC-SHA256(sk_live, t + ".POST." + path ของ webhook URL + "." + body)` แล้ว `v1 = HMAC-SHA256(key, t + "." + body)` (ค่าเริ่มต้นของทุกร้านตั้งแต่ 18 ต.ค. 2569 — ร้านที่ยังไม่มี secret key จะเซ็นด้วย webhook secret `whsec_…` แทน), `X-Webhook-Event: <event>` และ `User-Agent: OptionPay-Webhook/1.0` · ตอบ 2xx = สำเร็จ, ไม่สำเร็จจะลองใหม่ได้สูงสุด 5 ครั้ง (`payout.completed` / `payout.failed` ลองใหม่อัตโนมัติ ห่างกัน 5, 10, 15, 20 นาที)

GET /webhooks

รายการ Webhooks

สิทธิ์: **manage_webhooks** · จัดการ webhook

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792538"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/webhooks." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 77d18202808e54a1eb1ede2636a8bec3b0134229165d141cd8118fb343f9a64c

curl "https://optionpay.co/api/v1/webhooks" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/webhooks";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง:
```

```
"1791792538"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง:
"77d18202808e54a1eb1ede2636a8bec3b0134229165d141cd8118fb343f9a64c"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792538
สตริงที่เซ็น : 1791792538.GET./api/v1/webhooks.
X-Signature :
77d18202808e54a1eb1ede2636a8bec3b0134229165d141cd8118fb343f9a64c
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "data": []
}
```

POST /webhooks

เพิ่ม Webhook

สิทธิ์: **manage_webhooks** · จัดการ webhook

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

คำอธิบายฟิลด์

```
{ "url": "https://your-site.com/cb",   // https เท่านั้น
  "events": ["payment.approved", "payment.rejected"] }   // ทั้งหมด:
payment.approved · payment.rejected · payment.expired · payment.refunded ·
payout.completed · payout.failed · withdrawal.approved ·
withdrawal.completed · withdrawal.rejected
```

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792540"   # ปกติใช้ TS=$(date +%s)
BODY='{"url":"https://your-site.com/optionpay/webhook","events":["payment.approved","payment.rejected"]}'
SIG=$(printf '%s' "$TS.POST./api/v1/webhooks.$BODY" | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = ae635751e6749e04ab7dee5b169fc6fc27f2de4ca5e83ce8afeaf1cc488e09b9

curl -X POST "https://optionpay.co/api/v1/webhooks" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
  const path = "/api/v1/webhooks";
  const body = JSON.stringify({
    "url": "https://your-site.com/optionpay/webhook",
    "events": [
      "payment.approved",
      "payment.rejected"
    ]
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792540"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                                  // ค่าตัวอย่าง: "ae635751e6749e04ab7dee5b169fc6fc27f2de4ca5e83ce8afeaf1cc488e09b9"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
```

```
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792540
สตริงที่เซ็น : 1791792540.POST./api/v1/webhooks.{"url":"https://your-
site.com/optionpay/webhook","events":
["payment.approved","payment.rejected"]}
X-Signature :
ae635751e6749e04ab7dee5b169fc6fc27f2de4ca5e83ce8afeaf1cc488e09b9
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "webhook": {
    "id": "7",
    "url": "https://your-site.com/optionpay/webhook",
    "events": [
      "payment.approved",
      "payment.rejected"
    ],
    "status": "ACTIVE",
    "secret": "whsec_l3LRUXXEA3LYOxMWmsEM9rGiWrN1y_wL"
  }
}
```

Payload: `{ "event": "payment.approved", "ref": "TXN…", "orderId": "ORD-001", "amount": "100.07", "status": "approved", "createdAt": "…", "resentAt": "…" }` (`resentAt` = เวลาที่ส่ง) · สูงสุด 5 ปลายทางต่อร้าน · ไม่ยิงไปที่อยู่ภายใน (private IP) · **payment.expired / payment.refunded** ใช้ payload เดียวกับ payment.approved (status `expired` / `refunded`) · **withdrawal.approved / withdrawal.completed / withdrawal.rejected** ใช้รูปแบบเดียวกับ payout (`ref` = WDR..., status `approved` / `completed` / `failed` , `failureReason` = เหตุผลที่ปฏิเสธ) · ผู้ดูแลระบบเปิด/ปิดแต่ละ event ต่อปลายทางได้ · **`payout.completed`** — ทีม Option Pay โอนเงิน payout ที่สร้างผ่าน `POST /payout/create` ให้ผู้รับแล้ว (สถานะในหน้าร้าน: ถอนสำเร็จ) ส่งครั้งเดียวต่อรายการ: `{ "event": "payout.completed", "ref": "OUT10153", "orderId": "WD-…", "merchantRef": "WD-…", "amount": "450.00", "status": "completed", "failureReason": null, "createdAt": "2026-10-15T09:29:45.872Z", "completedAt": "2026-10-15T09:40:11.245Z" }` — `completedAt` = เวลาที่โอนเสร็จ · **`payout.failed`** — ผู้ดูแลระบบปฏิเสธ payout ที่สร้างผ่าน `POST /payout/create` (คืนยอด amount + fee เข้ายอดคงเหลือแล้ว): `{ "event": "payout.failed", "ref": "OUT10152", "orderId": "WD-…", "merchantRef": "WD-…", "amount": "550.00", "status": "failed", "failureReason": "ยกเลิกโดยผู้ดูแลระบบ", "createdAt": "2026-10-15T02:14:08.819Z", "completedAt": "2026-10-15T06:09:44.502Z" }` — `orderId` = `merchantRef` ที่ร้านส่งตอนสร้าง · `completedAt` = เวลาที่ปฏิเสธ · ตรงกับ `GET /payout/:id` (status `FAILED`) · payout ของร้านย่อยที่สร้างด้วย key ของร้านแม่ (`X-Sub-Merchant`) ส่งไปที่ webhook ของร้านที่ใกล้ที่สุดในสายที่สมัคร event นี้ไว้

Balance & Profile

GET `/balance`

ดูยอดเงินคงเหลือ

สิทธิ์: **view_dashboard** · ดูภาพรวม ยอดเงิน และข้อมูลร้าน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792541"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/balance." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = fab5caef11b00eabbc95f875c2216ad1a447b87dc44967d08fbae01200b087db

curl "https://optionpay.co/api/v1/balance" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/balance";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792541"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                                  // ค่าตัวอย่าง: "fab5caef11b00eabbc95f875c2216ad1a447b87dc44967d08fbae01200b087db"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
```

```
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792541
สตริงที่เซ็น : 1791792541.GET./api/v1/balance.
X-Signature :
fab5caef11b00eabbc95f875c2216ad1a447b87dc44967d08fbae01200b087db
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "balance": 1763.64,
  "availableBalance": 1630.64,
  "reserved": 133,
  "currency": "THB",
  "minBalance": 0,
  "minDepositAmount": 0,
  "minWithdrawAmount": 0
}
```

GET /balance/line-summary

ดูยอดรวม Line (Parent+Children)

สิทธิ์: **view_dashboard** · ดูภาพรวม ยอดเงิน และข้อมูลร้าน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMOshop00docsonly000000000000"   # ส่งไปกับคำขอ
SECRET="ss_live_DEMOshop00docsonly0000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791864965"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/balance/line-summary." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 417e1493637cfbbeeeb6521e89a63a08fc7f2c9096dabae83eb1c3309a72f028

curl "https://optionpay.co/api/v1/balance/line-summary" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMOshop00docsonly000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "ss_live_DEMOshop00docsonly00000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/balance/line-summary";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791864965"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง: "417e1493637cfbbeeeb6521e89a63a08fc7f2c9096dabae83eb1c3309a72f028"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791864965
สตริงที่เซ็น : 1791864965.GET./api/v1/balance/line-summary.
X-Signature :
417e1493637cfbbeeeb6521e89a63a08fc7f2c9096dabae83eb1c3309a72f028
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "parent": {
    "id": "28",
    "name": "Siam Holding",
    "balance": 10548.49
  },
  "children": [
```

```
    {
      "id": "29",
      "name": "ตัวแทนภาคเหนือ",
      "slug": "demo-north",
      "parentId": "28",
      "depth": 1,
      "status": "ACTIVE",
      "balance": 8654.4
    },
    {
      "id": "31",
      "name": "เชียงใหม่ สาขา 1",
      "slug": "demo-cm1",
      "parentId": "29",
      "depth": 2,
      "status": "ACTIVE",
      "balance": 14215.8
    },
    {
      "id": "33",
      "name": "ร้านนิมมาน",
      "slug": "demo-nimman",
      "parentId": "31",
      "depth": 3,
      "status": "ACTIVE",
      "balance": 20227.6
    },
    {
      "id": "34",
      "name": "ร้านสันติธรรม",
      "slug": "demo-santitham",
      "parentId": "31",
      "depth": 3,
      "status": "ACTIVE",
      "balance": 11469.6
    },
    {
      "id": "32",
      "name": "เชียงราย สาขา 1",
      "slug": "demo-cr1",
      "parentId": "29",
      "depth": 2,
      "status": "ACTIVE",
      "balance": 11712
    },
    {
      "id": "30",
      "name": "ตัวแทนภาคใต้",
      "slug": "demo-south",
      "parentId": "28",
      "depth": 1,
      "status": "ACTIVE",
```

```
      "balance": 7479.04
    },
    {
      "id": "35",
      "name": "ภูเก็ต สาขา 1",
      "slug": "demo-pk1",
      "parentId": "30",
      "depth": 2,
      "status": "ACTIVE",
      "balance": 15735.17
    },
    {
      "id": "36",
      "name": "ร้านป่าตอง",
      "slug": "demo-patong",
      "parentId": "35",
      "depth": 3,
      "status": "ACTIVE",
      "balance": 16949.59
    },
    {
      "id": "37",
      "name": "หาดใหญ่ สาขา 1",
      "slug": "demo-hy1",
      "parentId": "30",
      "depth": 2,
      "status": "PENDING",
      "balance": 0
    },
    {
      "id": "38",
      "name": "ร้านตัวอย่าง API",
      "slug": "demo-api-shop",
      "parentId": "28",
      "depth": 1,
      "status": "SUSPENDED",
      "balance": 0
    }
  ],
  "total": 116991.69,
  "currency": "THB"
}
```


children = ร้านย่อยทุกชั้นใต้ร้านนี้ (depth 1 = ชั้นถัดไป) · total = ยอดคงเหลือของร้านนี้ + ร้านย่อยทั้งหมด · ร้านที่ไม่มีร้านย่อยได้ `children: []`

GET /profile

ดูข้อมูล Merchant

สิทธิ์: **view_dashboard** · ดูภาพรวม ยอดเงิน และข้อมูลร้าน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792544"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/profile." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = d69985bee6822c7f1ab6828aadca5ba2a79fcb14573b6f9c38d5a7bb8cdf7f7c

curl "https://optionpay.co/api/v1/profile" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/profile";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792544"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                                  // ค่าตัวอย่าง: "d69985bee6822c7f1ab6828aadca5ba2a79fcb14573b6f9c38d5a7bb8cdf7f7c"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
```

```
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792544
สตริงที่เซ็น : 1791792544.GET./api/v1/profile.
X-Signature :
d69985bee6822c7f1ab6828aadca5ba2a79fcb14573b6f9c38d5a7bb8cdf7f7c
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "merchant": {
    "id": "6",
    "name": "your-shop",
    "slug": "your-shop",
    "status": "ACTIVE",
    "kycStatus": "pending",
    "feePercent": 1.5,
    "feeFlat": 0,
    "contactEmail": null,
    "contactPhone": null,
    "createdAt": "2026-10-04T07:51:13.753Z"
  }
}
```


GET /dashboard

Dashboard Stats

สิทธิ์: **view_dashboard** · ดูภาพรวม ยอดเงิน และข้อมูลร้าน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMO0000docsonly00000000000000000000000000"   # ส่งไปกับคำขอ
SECRET="sk_live_DEMO0000docsonly00000000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791792545"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/dashboard." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = f4ce7b4d0b089b7e24ac101568fe303ed6f847d720fdc1712dc256de874ec630

curl "https://optionpay.co/api/v1/dashboard" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMO0000docsonly00000000000000000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "sk_live_DEMO0000docsonly00000000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/dashboard";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791792545"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                                  // ค่าตัวอย่าง: "f4ce7b4d0b089b7e24ac101568fe303ed6f847d720fdc1712dc256de874ec630"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791792545
สตริงที่เซ็น : 1791792545.GET./api/v1/dashboard.
X-Signature :
f4ce7b4d0b089b7e24ac101568fe303ed6f847d720fdc1712dc256de874ec630
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "today": {
    "paidCount": 0,
    "paidAmount": 0,
    "fee": 0,
    "createdCount": 4,
    "payoutAmount": 0
```

```
  },
  "pending": {
    "payments": 3,
    "payouts": 3
  },
  "balance": 1763.64,
  "availableBalance": 1630.64
}
```

> “วันนี้” นับตามเวลาไทย (Asia/Bangkok)

Sub-merchants (ร้านย่อย)

ร้านสร้างร้านย่อยได้ไม่จำกัด และร้านย่อยสร้างร้านย่อยต่อได้ (เป็นต้นไม้) · ร้านย่อยใหม่มีสถานะ `PENDING` จนกว่าผู้ดูแลระบบอนุมัติ · อัตราค่าธรรมเนียมของร้านย่อยต้องไม่ต่ำกว่าของร้านแม่ — ร้านแม่ได้ส่วนต่างจากทุกรายการ (ดูคู่มือร้านค้าหัวข้อ ร้านย่อย) · อ่านข้อมูลได้ทุกชั้นใต้ร้าน แก้ไขได้เฉพาะร้านย่อยชั้นถัดไป · **X-Sub-Merchant**: ส่ง header `X-Sub-Merchant: <id>` พร้อม key ของร้านแม่ เพื่อเรียก endpoint ใดก็ได้ในนามร้านย่อยนั้น (ลายเซ็นเหมือนเดิม)

`GET` `/sub-merchants`

รายชื่อร้านย่อยทุกชั้นใต้ร้านนี้ พร้อมยอดคงเหลือ ยอดขายสะสม และค่าคอมที่ร้านนี้ได้จากแต่ละร้าน

สิทธิ์: **view_dashboard** · ดูภาพรวม ยอดเงิน และข้อมูลร้าน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMOshop00docsonly000000000000"   # ส่งไปกับคำขอ
SECRET="ss_live_DEMOshop00docsonly00000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791864962"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/sub-merchants." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 9af162c4aa35d277437faaa7f96240ebbd851b86fb526570c8644e8293889209

curl "https://optionpay.co/api/v1/sub-merchants" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMOshop00docsonly000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "ss_live_DEMOshop00docsonly00000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/sub-merchants";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791864962"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
```

```
    .digest("hex");                                          // ค่าตัวอย่าง:
"9af162c4aa35d277437faaa7f96240ebbd851b86fb526570c8644e8293889209"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791864962
สตริงที่เซ็น : 1791864962.GET./api/v1/sub-merchants.
X-Signature :
9af162c4aa35d277437faaa7f96240ebbd851b86fb526570c8644e8293889209
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "total": 9,
  "subMerchants": [
    {
      "id": "29",
      "name": "ตัวแทนภาคเหนือ",
      "slug": "demo-north",
      "parentId": "28",
      "depth": 1,
      "status": "ACTIVE",
      "kycStatus": "verified",
      "feePercent": 2,
      "feeFlat": 0,
      "outFeePercent": 0.8,
      "outFeeFlat": 5,
      "allowSubMerchants": true,
      "approvedAt": "2026-10-13T04:07:10.660Z",
      "createdAt": "2026-10-13T04:07:08.162Z",
      "contactPhone": null,
      "balance": 8654.4,
      "paidCount": 4,
      "paidTotal": 8500,
      "commissionToMe": 42.48,
      "childCount": 2,
      "owner": {
        "fullName": "เจ้าของ ตัวแทนภาคเหนือ",
        "email": "owner@demo-north.demo",
        "username": "demo-north"
```

```
      }
    },
    {
      "id": "31",
      "name": "เชียงใหม่ สาขา 1",
      "slug": "demo-cm1",
      "parentId": "29",
      "depth": 2,
      "status": "ACTIVE",
      "kycStatus": "verified",
      "feePercent": 2.5,
      "feeFlat": 0,
      "outFeePercent": 1,
      "outFeeFlat": 10,
      "allowSubMerchants": true,
      "approvedAt": "2026-10-13T04:07:18.512Z",
      "createdAt": "2026-10-13T04:07:15.913Z",
      "contactPhone": null,
      "balance": 14215.8,
      "paidCount": 8,
      "paidTotal": 14400,
      "commissionToMe": 72,
      "childCount": 2,
      "owner": {
        "fullName": "เจ้าของ เชียงใหม่ สาขา 1",
        "email": "owner@demo-cm1.demo",
        "username": "demo-cm1"
      }
    },
    {
      "id": "33",
      "name": "ร้านนิมมาน",
      "slug": "demo-nimman",
      "parentId": "31",
      "depth": 3,
      "status": "ACTIVE",
      "kycStatus": "verified",
      "feePercent": 3,
      "feeFlat": 0,
      "outFeePercent": 1.2,
      "outFeeFlat": 10,
      "allowSubMerchants": true,
      "approvedAt": "2026-10-13T04:07:26.309Z",
      "createdAt": "2026-10-13T04:07:23.813Z",
      "contactPhone": null,
      "balance": 20227.6,
      "paidCount": 15,
      "paidTotal": 26080,
      "commissionToMe": 145.4,
      "childCount": 0,
      "owner": {
        "fullName": "เจ้าของ ร้านนิมมาน",
```

```
        "email": "owner@demo-nimman.demo",
        "username": "demo-nimman"
      }
    },
    {
      "id": "34",
      "name": "ร้านสันติธรรม",
      "slug": "demo-santitham",
      "parentId": "31",
      "depth": 3,
      "status": "ACTIVE",
      "kycStatus": "verified",
      "feePercent": 2.8,
      "feeFlat": 0,
      "outFeePercent": 1,
      "outFeeFlat": 10,
      "allowSubMerchants": true,
      "approvedAt": "2026-10-13T04:07:30.211Z",
      "createdAt": "2026-10-13T04:07:27.759Z",
      "contactPhone": null,
      "balance": 11469.6,
      "paidCount": 12,
      "paidTotal": 11800,
      "commissionToMe": 59,
      "childCount": 0,
      "owner": {
        "fullName": "เจ้าของ ร้านสันติธรรม",
        "email": "owner@demo-santitham.demo",
        "username": "demo-santitham"
      }
    },
    {
      "id": "32",
      "name": "เชียงราย สาขา 1",
      "slug": "demo-cr1",
      "parentId": "29",
      "depth": 2,
      "status": "ACTIVE",
      "kycStatus": "verified",
      "feePercent": 2.4,
      "feeFlat": 0,
      "outFeePercent": 1,
      "outFeeFlat": 5,
      "allowSubMerchants": true,
      "approvedAt": "2026-10-13T04:07:22.415Z",
      "createdAt": "2026-10-13T04:07:19.968Z",
      "contactPhone": null,
      "balance": 11712,
      "paidCount": 8,
      "paidTotal": 12000,
      "commissionToMe": 59.96,
      "childCount": 0,
```

```
      "owner": {
        "fullName": "เจ้าของ เชียงราย สาขา 1",
        "email": "owner@demo-cr1.demo",
        "username": "demo-cr1"
      }
    },
    {
      "id": "30",
      "name": "ตัวแทนภาคใต้",
      "slug": "demo-south",
      "parentId": "28",
      "depth": 1,
      "status": "ACTIVE",
      "kycStatus": "verified",
      "feePercent": 1.8,
      "feeFlat": 0,
      "outFeePercent": 0.6,
      "outFeeFlat": 5,
      "allowSubMerchants": true,
      "approvedAt": "2026-10-13T04:07:14.508Z",
      "createdAt": "2026-10-13T04:07:12.059Z",
      "contactPhone": null,
      "balance": 7479.04,
      "paidCount": 4,
      "paidTotal": 7480,
      "commissionToMe": 22.44,
      "childCount": 2,
      "owner": {
        "fullName": "เจ้าของ ตัวแทนภาคใต้",
        "email": "owner@demo-south.demo",
        "username": "demo-south"
      }
    },
    {
      "id": "35",
      "name": "ภูเก็ต สาขา 1",
      "slug": "demo-pk1",
      "parentId": "30",
      "depth": 2,
      "status": "ACTIVE",
      "kycStatus": "verified",
      "feePercent": 2.2,
      "feeFlat": 0,
      "outFeePercent": 0.8,
      "outFeeFlat": 5,
      "allowSubMerchants": true,
      "approvedAt": "2026-10-13T04:07:34.061Z",
      "createdAt": "2026-10-13T04:07:31.609Z",
      "contactPhone": null,
      "balance": 15735.17,
      "paidCount": 10,
      "paidTotal": 16000,
```

```
      "commissionToMe": 48,
      "childCount": 1,
      "owner": {
        "fullName": "เจ้าของ ภูเก็ต สาขา 1",
        "email": "owner@demo-pk1.demo",
        "username": "demo-pk1"
      }
    },
    {
      "id": "36",
      "name": "ร้านป่าตอง",
      "slug": "demo-patong",
      "parentId": "35",
      "depth": 3,
      "status": "ACTIVE",
      "kycStatus": "verified",
      "feePercent": 2.7,
      "feeFlat": 0,
      "outFeePercent": 1,
      "outFeeFlat": 10,
      "allowSubMerchants": true,
      "approvedAt": "2026-10-13T04:07:38.019Z",
      "createdAt": "2026-10-13T04:07:35.590Z",
      "contactPhone": null,
      "balance": 16949.59,
      "paidCount": 14,
      "paidTotal": 17420,
      "commissionToMe": 52.21,
      "childCount": 0,
      "owner": {
        "fullName": "เจ้าของ ร้านป่าตอง",
        "email": "owner@demo-patong.demo",
        "username": "demo-patong"
      }
    },
    {
      "id": "37",
      "name": "หาดใหญ่ สาขา 1",
      "slug": "demo-hy1",
      "parentId": "30",
      "depth": 2,
      "status": "PENDING",
      "kycStatus": "pending",
      "feePercent": 2.3,
      "feeFlat": 0,
      "outFeePercent": 0.8,
      "outFeeFlat": 5,
      "allowSubMerchants": true,
      "approvedAt": null,
      "createdAt": "2026-10-13T04:07:39.408Z",
      "contactPhone": null,
      "balance": 0,
```

```
      "paidCount": 0,
      "paidTotal": 0,
      "commissionToMe": 0,
      "childCount": 0,
      "owner": {
        "fullName": "เจ้าของ หาดใหญ่ สาขา 1",
        "email": "owner@demo-hy1.demo",
        "username": "demo-hy1"
      }
    }
  ]
}
```

GET **/sub-merchants/overview**

ภาพรวมร้านย่อยทั้งหมด (หรือร้านเดียวด้วย ?shop=<id>) — ยอดรวม แนวโน้มรายวัน และตัวเลขรายร้าน · ?days=7|30|90 (ค่าเริ่มต้น 30)

สิทธิ์: **view_dashboard** · ดูภาพรวม ยอดเงิน และข้อมูลร้าน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMOshop00docsonly000000000000"   # ส่งไปกับคำขอ
SECRET="ss_live_DEMOshop00docsonly00000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791864963"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/sub-merchants/overview?days=7." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = f2eed18cad8b48dc31fe3071720e9c51fa1c476f8cfaf104f457d5042d0db3ac

curl "https://optionpay.co/api/v1/sub-merchants/overview?days=7" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMOshop00docsonly000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "ss_live_DEMOshop00docsonly00000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
```

```
  const path = "/api/v1/sub-merchants/overview?days=7";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง:
"1791864963"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                              // ค่าตัวอย่าง:
"f2eed18cad8b48dc31fe3071720e9c51fa1c476f8cfaf104f457d5042d0db3ac"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791864963
สตริงที่เซ็น : 1791864963.GET./api/v1/sub-merchants/overview?days=7.
X-Signature :
f2eed18cad8b48dc31fe3071720e9c51fa1c476f8cfaf104f457d5042d0db3ac
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "days": 7,
  "shop": null,
  "totals": {
    "shops": 9,
    "active": 8,
    "pending": 1,
    "suspended": 0,
    "paidAmount": 32550,
    "paidCount": 19,
    "feesPaid": 864.42,
    "outAmount": 5000,
    "outCount": 1,
    "outFees": 70,
    "commissionToMe": 159.36,
    "balance": 106443.2,
    "pendingPayments": 1,
    "pendingWithdrawals": 3000
  },
  "trend": [
```

```
    {
      "day": "2026-10-07",
      "incoming": 1520,
      "outgoing": 0,
      "commission": 4.56,
      "count": 1
    },
    {
      "day": "2026-10-08",
      "incoming": 3325,
      "outgoing": 0,
      "commission": 14.16,
      "count": 3
    },
    {
      "day": "2026-10-09",
      "incoming": 4125,
      "outgoing": 0,
      "commission": 20.62,
      "count": 2
    },
    {
      "day": "2026-10-10",
      "incoming": 6375,
      "outgoing": 0,
      "commission": 24.62,
      "count": 4
    },
    {
      "day": "2026-10-11",
      "incoming": 0,
      "outgoing": 0,
      "commission": 0,
      "count": 0
    },
    {
      "day": "2026-10-12",
      "incoming": 7205,
      "outgoing": 0,
      "commission": 30.4,
      "count": 8
    },
    {
      "day": "2026-10-13",
      "incoming": 10000,
      "outgoing": 5000,
      "commission": 65,
      "count": 1
    }
  ],
  "shops": [
    {
```

```
      "id": "29",
      "name": "ตัวแทนภาคเหนือ",
      "slug": "demo-north",
      "parentId": "28",
      "depth": 1,
      "status": "ACTIVE",
      "paidAmount": 1375,
      "paidCount": 1,
      "feesPaid": 27.5,
      "outAmount": 0,
      "outCount": 0,
      "outFees": 0,
      "commissionToMe": 6.87,
      "balance": 8654.4,
      "pendingPayments": 0,
      "pendingWithdrawals": 0
    },
    {
      "id": "31",
      "name": "เชียงใหม่ สาขา 1",
      "slug": "demo-cm1",
      "parentId": "29",
      "depth": 2,
      "status": "ACTIVE",
      "paidAmount": 3240,
      "paidCount": 2,
      "feesPaid": 81,
      "outAmount": 0,
      "outCount": 0,
      "outFees": 0,
      "commissionToMe": 16.2,
      "balance": 14215.8,
      "pendingPayments": 0,
      "pendingWithdrawals": 0
    },
    {
      "id": "33",
      "name": "ร้านนิมมาน",
      "slug": "demo-nimman",
      "parentId": "31",
      "depth": 3,
      "status": "ACTIVE",
      "paidAmount": 13300,
      "paidCount": 4,
      "feesPaid": 399,
      "outAmount": 5000,
      "outCount": 1,
      "outFees": 70,
      "commissionToMe": 81.5,
      "balance": 20227.6,
      "pendingPayments": 0,
      "pendingWithdrawals": 0
```

```
    },
    {
      "id": "34",
      "name": "ร้านสันติธรรม",
      "slug": "demo-santitham",
      "parentId": "31",
      "depth": 3,
      "status": "ACTIVE",
      "paidAmount": 2750,
      "paidCount": 3,
      "feesPaid": 77,
      "outAmount": 0,
      "outCount": 0,
      "outFees": 0,
      "commissionToMe": 13.75,
      "balance": 11469.6,
      "pendingPayments": 0,
      "pendingWithdrawals": 3000
    },
    {
      "id": "32",
      "name": "เชียงราย สาขา 1",
      "slug": "demo-cr1",
      "parentId": "29",
      "depth": 2,
      "status": "ACTIVE",
      "paidAmount": 2700,
      "paidCount": 2,
      "feesPaid": 64.8,
      "outAmount": 0,
      "outCount": 0,
      "outFees": 0,
      "commissionToMe": 13.49,
      "balance": 11712,
      "pendingPayments": 0,
      "pendingWithdrawals": 0
    },
    {
      "id": "30",
      "name": "ตัวแทนภาคใต้",
      "slug": "demo-south",
      "parentId": "28",
      "depth": 1,
      "status": "ACTIVE",
      "paidAmount": 1210,
      "paidCount": 1,
      "feesPaid": 21.78,
      "outAmount": 0,
      "outCount": 0,
      "outFees": 0,
      "commissionToMe": 3.63,
      "balance": 7479.04,
```

```
      "pendingPayments": 0,
      "pendingWithdrawals": 0
    },
    {
      "id": "35",
      "name": "ภูเก็ต สาขา 1",
      "slug": "demo-pk1",
      "parentId": "30",
      "depth": 2,
      "status": "ACTIVE",
      "paidAmount": 4400,
      "paidCount": 3,
      "feesPaid": 96.8,
      "outAmount": 0,
      "outCount": 0,
      "outFees": 0,
      "commissionToMe": 13.2,
      "balance": 15735.17,
      "pendingPayments": 0,
      "pendingWithdrawals": 0
    },
    {
      "id": "36",
      "name": "ร้านป่าตอง",
      "slug": "demo-patong",
      "parentId": "35",
      "depth": 3,
      "status": "ACTIVE",
      "paidAmount": 3575,
      "paidCount": 3,
      "feesPaid": 96.54,
      "outAmount": 0,
      "outCount": 0,
      "outFees": 0,
      "commissionToMe": 10.72,
      "balance": 16949.59,
      "pendingPayments": 1,
      "pendingWithdrawals": 0
    },
    {
      "id": "37",
      "name": "หาดใหญ่ สาขา 1",
      "slug": "demo-hy1",
      "parentId": "30",
      "depth": 2,
      "status": "PENDING",
      "paidAmount": 0,
      "paidCount": 0,
      "feesPaid": 0,
      "outAmount": 0,
      "outCount": 0,
      "outFees": 0,
```

```
      "commissionToMe": 0,
      "balance": 0,
      "pendingPayments": 0,
      "pendingWithdrawals": 0
    }
  ]
}
```

ยอดในช่วง (paid / out / commission) นับรายการที่อนุมัติใน N วันล่าสุดตามเวลาไทย · balance และ pending เป็นข้อมูลตอนนี้

GET /sub-merchants/:id

ร้านย่อยร้านเดียว (ชั้นใดก็ได้ใต้ร้านนี้) พร้อมยอดรวมและแนวโน้มของร้านนั้น · ?days=7|30|90

สิทธิ์: **view_dashboard** · ดูภาพรวม ยอดเงิน และข้อมูลร้าน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMOshop00docsonly000000000000"   # ส่งไปกับคำขอ
SECRET="ss_live_DEMOshop00docsonly0000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791864963"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/sub-merchants/33?days=7." | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 46c4427fc9f6c9456344ad72a63682bda616093925f7409bb3b5ca157e3d9635

curl "https://optionpay.co/api/v1/sub-merchants/33?days=7" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMOshop00docsonly000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "ss_live_DEMOshop00docsonly0000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/sub-merchants/33?days=7";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง:
```

```
"1791864963"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง:
"46c4427fc9f6c9456344ad72a63682bda616093925f7409bb3b5ca157e3d9635"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791864963
สตริงที่เซ็น : 1791864963.GET./api/v1/sub-merchants/33?days=7.
X-Signature :
46c4427fc9f6c9456344ad72a63682bda616093925f7409bb3b5ca157e3d9635
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "subMerchant": {
    "id": "33",
    "name": "ร้านนิมมาน",
    "slug": "demo-nimman",
    "parentId": "31",
    "depth": 3,
    "status": "ACTIVE",
    "kycStatus": "verified",
    "feePercent": 3,
    "feeFlat": 0,
    "outFeePercent": 1.2,
    "outFeeFlat": 10,
    "allowSubMerchants": true,
    "approvedAt": "2026-10-13T04:07:26.309Z",
    "createdAt": "2026-10-13T04:07:23.813Z",
    "contactPhone": null,
    "balance": 20227.6,
    "paidCount": 15,
    "paidTotal": 26080,
    "commissionToMe": 145.4,
    "childCount": 0,
    "owner": {
```

```
      "fullName": "เจ้าของ ร้านนิมมาน",
      "email": "owner@demo-nimman.demo",
      "username": "demo-nimman"
    }
  },
  "overview": {
    "days": 7,
    "totals": {
      "shops": 1,
      "active": 1,
      "pending": 0,
      "suspended": 0,
      "paidAmount": 13300,
      "paidCount": 4,
      "feesPaid": 399,
      "outAmount": 5000,
      "outCount": 1,
      "outFees": 70,
      "commissionToMe": 81.5,
      "balance": 20227.6,
      "pendingPayments": 0,
      "pendingWithdrawals": 0
    },
    "trend": [
      {
        "day": "2026-10-07",
        "incoming": 0,
        "outgoing": 0,
        "commission": 0,
        "count": 0
      },
      {
        "day": "2026-10-08",
        "incoming": 1140,
        "outgoing": 0,
        "commission": 5.7,
        "count": 1
      },
      {
        "day": "2026-10-09",
        "incoming": 0,
        "outgoing": 0,
        "commission": 0,
        "count": 0
      },
      {
        "day": "2026-10-10",
        "incoming": 1500,
        "outgoing": 0,
        "commission": 7.5,
        "count": 1
      },
```

```
      {
        "day": "2026-10-11",
        "incoming": 0,
        "outgoing": 0,
        "commission": 0,
        "count": 0
      },
      {
        "day": "2026-10-12",
        "incoming": 660,
        "outgoing": 0,
        "commission": 3.3,
        "count": 1
      },
      {
        "day": "2026-10-13",
        "incoming": 10000,
        "outgoing": 5000,
        "commission": 65,
        "count": 1
      }
    ]
  }
}
```


POST /sub-merchants

สร้างร้านย่อยใต้ร้านนี้ พร้อมผู้ใช้เจ้าของร้าน (เข้าสู่ระบบ Merchant Portal ด้วยอีเมล + รหัสผ่านชั่วคราว)

สิทธิ์: **manage_sub_merchants** · สร้าง / จัดการร้านย่อย

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

คำอธิบายฟิลด์

```
{
  "name": "ร้านนิมมาน",                    // required
  "slug": "nimman",                        // required — a–z 0–9 และ - (แก้ภายหลังไม่ได้)
  "feePercent": 3, "feeFlat": 0,           // ค่าธรรมเนียมเงินเข้า — ต้องไม่ต่ำกว่าของร้านนี้
  "outFeePercent": 1.2, "outFeeFlat": 10,  // ค่าธรรมเนียมเงินออก — ต้องไม่ต่ำกว่าของร้านนี้
  "allowSubMerchants": true,               // ร้านนี้สร้างร้านย่อยต่อได้ไหม (ค่าเริ่มต้น true)
  "contactPhone": "053-000-000",           // optional
  "owner": { "fullName": "คุณเจ้าของ", "email": "owner@nimman.example", "password": "รหัสชั่วคราว ≥ 8 ตัว" }
}
```

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMOshop00docsonly000000000000"   # ส่งไปกับคำขอ
SECRET="ss_live_DEMOshop00docsonly00000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791864964"   # ปกติใช้ TS=$(date +%s)
BODY='{"name":"ร้านตัวอย่าง API","slug":"demo-api-shop","feePercent":2.2,"feeFlat":0,"outFeePercent":0.8,"outFeeFlat":5,"allowSubMerchants":false,"contactPhone":"02-000-2000","owner":{"fullName":"คุณตัวอย่าง เอพีไอ","email":"owner@demo-api-shop.demo","password":"Temp#2026Api"}}'
SIG=$(printf '%s' "$TS.POST./api/v1/sub-merchants.$BODY" | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 48c68feab3eb73b8f5502baa6709faed9ef3e5dc8a1360e3dd2be85c0ddb2b09

curl -X POST "https://optionpay.co/api/v1/sub-merchants" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMOshop00docsonly000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "ss_live_DEMOshop00docsonly00000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
  const path = "/api/v1/sub-merchants";
  const body = JSON.stringify({
    "name": "ร้านตัวอย่าง API",
    "slug": "demo-api-shop",
    "feePercent": 2.2,
    "feeFlat": 0,
    "outFeePercent": 0.8,
    "outFeeFlat": 5,
    "allowSubMerchants": false,
    "contactPhone": "02-000-2000",
    "owner": {
      "fullName": "คุณตัวอย่าง เอพีไอ",
      "email": "owner@demo-api-shop.demo",
      "password": "Temp#2026Api"
    }
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791864964"
```

```
  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง:
"48c68feab3eb73b8f5502baa6709faed9ef3e5dc8a1360e3dd2be85c0ddb2b09"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791864964
สตริงที่เซ็น : 1791864964.POST./api/v1/sub-merchants.{"name":"ร้านตัวอย่าง
API","slug":"demo-api-
shop","feePercent":2.2,"feeFlat":0,"outFeePercent":0.8,"outFeeFlat":5,"allo
wSubMerchants":false,"contactPhone":"02-000-2000","owner":{"fullName":"คุณ
ตัวอย่าง เอพีไอ","email":"owner@demo-api-
shop.demo","password":"Temp#2026Api"}}
X-Signature :
48c68feab3eb73b8f5502baa6709faed9ef3e5dc8a1360e3dd2be85c0ddb2b09
```

ผลลัพธ์ — HTTP 200


```
{
  "success": true,
  "id": 38,
  "owner": {
    "username": "demo-api-shop",
    "email": "owner@demo-api-shop.demo"
  },
  "status": "PENDING"
}
```


Errors: `400 RATE_BELOW_PARENT` อัตราต่ำกว่าร้านแม่ · `409 SLUG_TAKEN` · `409 EMAIL_TAKEN` (1 อีเมลใช้ได้กับร้านเดียว) · `403 SUB_MERCHANTS_NOT_ALLOWED` · `403 SHOP_NOT_ACTIVE` ร้านนี้ยังรออนุมัติ · `400 TOO_DEEP` ลึกเกิน 10 ชั้น

POST `/sub-merchants/:id`

แก้ร้านย่อยชั้นถัดไป: ชื่อ อัตราค่าธรรมเนียม สถานะ (ACTIVE / SUSPENDED) และสิทธิ์สร้างร้านย่อยต่อ

สิทธิ์: **manage_sub_merchants** · สร้าง / จัดการร้านย่อย

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

คำอธิบายฟิลด์

```
{ "name": "ร้านนิมมาน", "feePercent": 3, "feeFlat": 0, "outFeePercent": 1.2,
"outFeeFlat": 10, "allowSubMerchants": true, "status": "suspended" }
```

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMOshop00docsonly000000000000"   # ส่งไปกับคำขอ
SECRET="ss_live_DEMOshop00docsonly0000000000000000000000"   # ใช้เซ็นเท่านั้น ห้ามส่ง
TS="1791864964"   # ปกติใช้ TS=$(date +%s)
BODY='{"name":"ร้านตัวอย่าง API","feePercent":2.4,"feeFlat":0,"outFeePercent":0.8,"outFeeFlat":5,"allowSubMerchants":false,"status":"suspended"}'
SIG=$(printf '%s' "$TS.POST./api/v1/sub-merchants/38.$BODY" | openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 258b7435b09aceb0d5639d112c838f25f29cd3bf0414d91e6df652de3048fc89

curl -X POST "https://optionpay.co/api/v1/sub-merchants/38" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMOshop00docsonly000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "ss_live_DEMOshop00docsonly0000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
  const path = "/api/v1/sub-merchants/38";
  const body = JSON.stringify({
    "name": "ร้านตัวอย่าง API",
    "feePercent": 2.4,
    "feeFlat": 0,
    "outFeePercent": 0.8,
```

```
    "outFeeFlat": 5,
    "allowSubMerchants": false,
    "status": "suspended"
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง:
"1791864964"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง:
"258b7435b09aceb0d5639d112c838f25f29cd3bf0414d91e6df652de3048fc89"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791864964
สตริงที่เซ็น : 1791864964.POST./api/v1/sub-merchants/38.{"name":"ร้านตัวอย่าง
API","feePercent":2.4,"feeFlat":0,"outFeePercent":0.8,"outFeeFlat":5,"allow
SubMerchants":false,"status":"suspended"}
X-Signature :
258b7435b09aceb0d5639d112c838f25f29cd3bf0414d91e6df652de3048fc89
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "status": "suspended"
}
```

> ระงับร้าน = ร้านย่อยทุกชั้นใต้ร้านนั้นใช้งานไม่ได้ด้วย · ร้านที่ยังรออนุมัติเปิดใช้งานเองไม่ได้ (`403 AWAITING_APPROVAL`) · อัตราต้องไม่สูงกว่าร้านย่อยของร้านนั้น (`400 RATE_ABOVE_CHILD`) · แก้ร้านในชั้นลึกกว่าได้ไม่ได้ (`404 SUB_MERCHANT_NOT_FOUND`)

POST /sub-merchants/:id/owner-password

ตั้งรหัสผ่านชั่วคราวใหม่ให้เจ้าของร้านย่อยชั้นถัดไป (ต้องเปลี่ยนเมื่อเข้าสู่ระบบครั้งถัดไป)

สิทธิ์: **manage_sub_merchants** · สร้าง / จัดการร้านย่อย

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature`

คำอธิบายฟิลด์

```
{ "password": "รหัสชั่วคราว ≥ 8 ตัว" }
```

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMOshop00docsonly000000000000"   # ส่งไปกับคำขอ
SECRET="ss_live_DEMOshop00docsonly00000000000000000000000"   # ใช้เซ็นเท่านั้น
ห้ามส่ง
TS="1791864965"   # ปกติใช้ TS=$(date +%s)
BODY='{"password":"NewTemp#2026"}'
SIG=$(printf '%s' "$TS.POST./api/v1/sub-merchants/38/owner-password.$BODY"
| openssl dgst -sha256 -hmac "$SECRET" | awk '{print $NF}')
# SIG = 65b15efec5084716d4a9ff3a29b6e573941b17a4910ee895ea1fe31e5c356eba

curl -X POST "https://optionpay.co/api/v1/sub-merchants/38/owner-password"
\
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  -H "Content-Type: application/json" \
  --data-raw "$BODY"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น
ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง:
"ak_live_DEMOshop00docsonly000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง:
"ss_live_DEMOshop00docsonly00000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function POST() {
  const method = "POST";
  const path = "/api/v1/sub-merchants/38/owner-password";
  const body = JSON.stringify({
    "password": "NewTemp#2026"
  });
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง:
"1791864965"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง:
```

```
"65b15efec5084716d4a9ff3a29b6e573941b17a4910ee895ea1fe31e5c356eba"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "Content-Type": "application/json",
    },
    body, // ต้องส่งสตริงเดียวกับที่เซ็นทุกตัวอักษร
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791864965
สตริงที่เซ็น : 1791864965.POST./api/v1/sub-merchants/38/owner-password.
{"password":"NewTemp#2026"}
X-Signature :
65b15efec5084716d4a9ff3a29b6e573941b17a4910ee895ea1fe31e5c356eba
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "owner": {
    "username": "demo-api-shop",
    "email": "owner@demo-api-shop.demo"
  }
}
```

GET /balance (X-Sub-Merchant)

ตัวอย่าง X-Sub-Merchant: key ของร้านแม่ + header X-Sub-Merchant เรียก /balance ของร้านย่อย

สิทธิ์: **view_dashboard** · ดูภาพรวม ยอดเงิน และข้อมูลร้าน

AUTH: `X-API-Key` + `X-Timestamp` + `X-Signature` + `X-Sub-Merchant`

ตัวอย่าง — CURL (เซ็นด้วย OPENSSL)

```
API_KEY="ak_live_DEMOshop00docsonly000000000000"   # ส่งไปกับคำขอ
SECRET="ss_live_DEMOshop00docsonly0000000000000000000000"   # ใช้เซ็นเท่านั้น
ห้ามส่ง
TS="1791864965"   # ปกติใช้ TS=$(date +%s)
SIG=$(printf '%s' "$TS.GET./api/v1/balance." | openssl dgst -sha256 -hmac
"$SECRET" | awk '{print $NF}')
# SIG = b3ee5a803e98110cb1bc1766d9e2b70626f75a389dc3b57ee9ab41aebdf8b8ba
```

```
curl "https://optionpay.co/api/v1/balance" \
  -H "X-API-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Sub-Merchant: 33" \
  -H "X-Signature: $SIG"
```

ตัวอย่าง — NEXT.JS (ROUTE HANDLER · สร้างลายเซ็น)

```
// app/api/optionpay/route.ts — Next.js route handler (ทำงานฝั่งเซิร์ฟเวอร์เท่านั้น ห้ามใช้ในหน้าเว็บของลูกค้า)
import { createHmac } from "node:crypto";

const API_KEY = process.env.OPTIONPAY_API_KEY!;    // ค่าตัวอย่าง: "ak_live_DEMOshop00docsonly000000000000"
const SECRET = process.env.OPTIONPAY_SECRET_KEY!;  // ค่าตัวอย่าง: "ss_live_DEMOshop00docsonly00000000000000000000000" — ห้ามส่งไปกับคำขอ

export async function GET() {
  const method = "GET";
  const path = "/api/v1/balance";
  const body = ""; // GET ไม่มี body — ใช้สตริงว่างตอนเซ็น
  const timestamp = Math.floor(Date.now() / 1000).toString();  // ค่าตัวอย่าง: "1791864965"

  // ลายเซ็น = HMAC-SHA256(secret, "timestamp.METHOD.path.body") แบบ hex
  const signature = createHmac("sha256", SECRET)
    .update(`${timestamp}.${method}.${path}.${body}`)
    .digest("hex");                                          // ค่าตัวอย่าง: "b3ee5a803e98110cb1bc1766d9e2b70626f75a389dc3b57ee9ab41aebdf8b8ba"

  const res = await fetch(`https://optionpay.co${path}`, {
    method,
    headers: {
      "X-API-Key": API_KEY,
      "X-Timestamp": timestamp,
      "X-Signature": signature,
      "X-Sub-Merchant": "33",
    },
  });
  return Response.json(await res.json(), { status: res.status });
}
```

ค่าในตัวอย่าง

```
X-Timestamp : 1791864965
สตริงที่เซ็น : 1791864965.GET./api/v1/balance.
X-Signature :
b3ee5a803e98110cb1bc1766d9e2b70626f75a389dc3b57ee9ab41aebdf8b8ba
```

ผลลัพธ์ — HTTP 200

```
{
  "success": true,
  "balance": 20227.6,
  "availableBalance": 20227.6,
  "reserved": 0,
  "currency": "THB",
  "minBalance": 0,
  "minDepositAmount": 0,
  "minWithdrawAmount": 0
}
```


ใช้ได้กับทุก endpoint · ร้านต้องอยู่ใต้ร้านของ key (`403 SUB_MERCHANT_NOT_FOUND`) และเปิดใช้งาน (`403 SUB_MERCHANT_NOT_ACTIVE`) · สิทธิ์ตามระดับผู้ใช้ของ key เดิม